HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

o0o

PROJECT 01

MÔN PHÁT TRIỂN HỆ THỐNG THÔNG MINH

ĐỀ TÀI

HỆ ĐA AGENT CHO BỆNH TIỂU ĐƯỜNG

Deep Learning, Knowledge Base và RAG, Reinforcement Learning, luật an toàn và demo phòng khám AI 3D trên localhost

Giảng viên hướng dẫn:	PGS.TS. Trần Đình Quế
Sinh viên thực hiện:	Nguyễn Thành Trung
Mã sinh viên:	B23DCCN861
Lớp:	D23CTPM01-B
Nhóm lớp:	CT
Nhóm:	07

Hà Nội, tháng 11 năm 2026

MỤC LỤC

1. Giới thiệu 4
1.1. Bài toán và động lực 4
1.2. Mục tiêu và phạm vi 4
1.3. Đóng góp và cấu trúc báo cáo 4
2. Hệ đa agent 5
2.1. Agent 5
2.2. Hệ đa agent 5
3. Kiến trúc tổng quan 6
3.1. Vai trò của các agent 7
3.2. Luồng một lượt hội chẩn 7
3.3. Công nghệ và triển khai 8
4. Các thành phần của hệ thống 9
4.1. Deep Learning ⇒ Agent 9
4.2. Knowledge Base và RAG 12
4.3. Reinforcement Learning: huấn luyện viên lối sống 14
4.4. Agent luật an toàn 15
4.5. Bác sĩ trưởng và báo cáo tự sinh 16
5. Code và Demo 17
5.1. Cấu trúc mã nguồn 17
5.2. Các hàm chính 17
5.3. Cách chạy 19
5.4. Demo: phòng khám AI 3D 19
5.5. Notebook thực hành 23
5.6. Nâng cấp demo: phản biện, giải thích, hỏi đáp, độ tin cậy 27
5.7. Kiểm thử 33
5.8. Đánh giá và hạn chế 33
5.9. Hướng phát triển 33
6. Kết luận 34
Tài liệu tham khảo 34

1. Giới thiệu

1.1. Bài toán và động lực

Bệnh tiểu đường xảy ra khi đường huyết (glucose) trong máu quá cao vì cơ thể thiếu insulin hoặc không dùng insulin hiệu quả. Theo tài liệu của Viện Quốc gia về Đái tháo đường, Tiêu hóa và Bệnh thận Hoa Kỳ (NIDDK), khoảng 90–95% ca bệnh là tiểu đường típ 2, loại có thể trì hoãn hoặc phòng ngừa nếu phát hiện sớm và thay đổi lối sống [4]. Điểm khó là bệnh tiến triển chậm: nhiều người không có triệu chứng rõ trong nhiều năm.

Một công cụ hỗ trợ tốt cần làm được bốn việc cùng lúc: dự đoán nguy cơ từ số liệu sức khỏe, giải thích dựa trên kiến thức y khoa có nguồn, gợi ý hành động cụ thể mỗi ngày, và cảnh báo khi có dấu hiệu nguy hiểm. Mỗi việc cần một kỹ thuật khác nhau, nên project này xây dựng một hệ đa agent: mỗi agent đảm nhận một vai và một bộ điều phối ghép các ý kiến thành lời tư vấn dễ hiểu.

1.2. Mục tiêu và phạm vi

- **Dự đoán và phân loại:** huấn luyện mạng nơ-ron trên 253.680 hồ sơ khảo sát sức khỏe để ước lượng xác suất ba nhóm (không, tiền tiểu đường, tiểu đường).
- **Knowledge Base và RAG:** dựng đồ thị tri thức 41 nút, 54 cạnh từ tài liệu NIDDK và truy xuất đoạn trích có nguồn.
- **Reinforcement learning:** huấn luyện viên học cách chọn bữa ăn và vận động cho bệnh nhân ảo để giữ đường huyết trong ngưỡng an toàn.
- **Luật an toàn:** agent dùng luật cố định để cảnh báo khẩn, luôn được ưu tiên.
- **Code và demo:** phòng khám AI 3D chạy trên localhost, người dùng đóng vai bệnh nhân; hệ tự sinh hình và bảng báo cáo.

Lưu ý. Phạm vi: đây là sản phẩm học tập. Hệ thống không chẩn đoán, không thay bác sĩ, và mô phỏng 24 giờ là mô hình đồ chơi do nhóm tự xây dựng, không phải mô hình y khoa. Mọi con số kết quả trong báo cáo được đo lại từ mã trong thư mục project01.

1.3. Đóng góp và cấu trúc báo cáo

- Một hệ đa agent hoàn chỉnh gồm bảy agent (sáu agent chuyên môn và một bác sĩ phản biện), kết nối bằng một giao thức sự kiện thống nhất.
- Agent Deep Learning trả về cả giải thích (yếu tố tác động) và what-if, không chỉ một con số.
- Graph-RAG: dùng đồ thị tri thức để cộng điểm cho đoạn trích, đo trên bộ 16 câu hỏi có đáp án.
- Môi trường mô phỏng 24 giờ và huấn luyện viên RL so sánh với luật và với trường hợp không có huấn luyện viên.
- Phòng khám 3D cho người dùng nhập vai, kèm báo cáo tự sinh gồm hình và bảng.

Phần 2 trình bày cơ sở về agent và hệ đa agent; phần 3 là kiến trúc tổng quan; phần 4 mô tả từng thành phần (Deep Learning, KB và RAG, RL, luật, điều phối); phần 5 trình bày mã chính, demo và kiểm thử; cuối cùng là đánh giá, hạn chế và kết luận.

2. Hệ đa agent

2.1. Agent

Agent là một chương trình nhận thông tin từ môi trường (quan sát), chọn hành động và theo đuổi một mục tiêu trong phạm vi được giao [1]–[3]. Điều quan trọng là vòng lặp quan sát, quyết định, hành động và nhận phản hồi, chứ không chỉ là một câu trả lời nghe hợp lý.



Hình 1. Vòng lặp của một agent trong bối cảnh tư vấn sức khỏe.

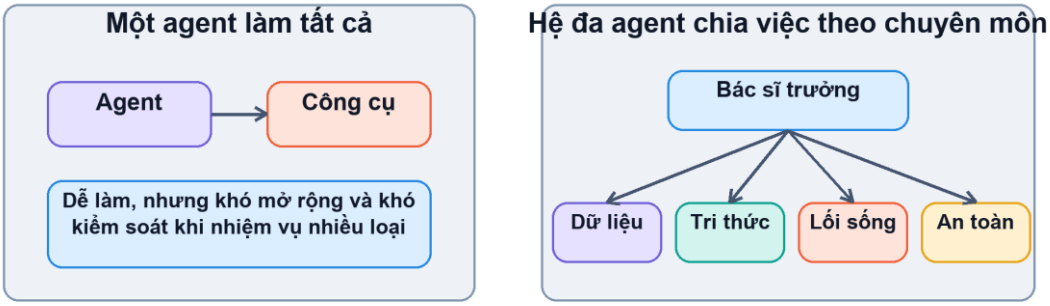
Theo cách phân loại kinh điển của Russell và Norvig [7], các agent khác nhau ở cách ra quyết định. Bảng 1 đặt bốn loại agent của project vào cách phân loại đó.

Bảng 1. Các loại agent trong hệ thống.

Loại agent	Cách ra quyết định	Trong project
Agent học (learning agent)	Cải thiện lựa chọn từ kinh nghiệm và phần thưởng	Huấn luyện viên RL; mạng nơ-ron học từ dữ liệu BRFSS
Agent dựa trên mục tiêu / hiểu biết	Dùng mô hình thế giới và tri thức để tìm câu trả lời	Thủ thư tri thức (đồ thị + RAG)
Agent phản xạ theo luật (rule-based)	Nếu điều kiện thì hành động; không học	Hộ lý an toàn
Agent điều phối	Chia việc, gom kết quả, quyết định bước tiếp theo	Bác sĩ trưởng

2.2. Hệ đa agent

Một agent duy nhất làm tất cả sẽ phải pha trộn dữ liệu số, tài liệu văn bản, mô phỏng và luật an toàn trong cùng một nơi, rất khó kiểm tra. Chia theo chuyên môn giúp mỗi agent đơn giản, kiểm thử riêng được, và cho phép thay một agent mà không đụng các agent khác. Đổi lại cần một giao thức phối hợp và phải kiểm soát việc các agent có thể mâu thuẫn nhau [5].



Hình 2. Một agent làm tất cả và hệ đa agent chia việc theo chuyên môn.

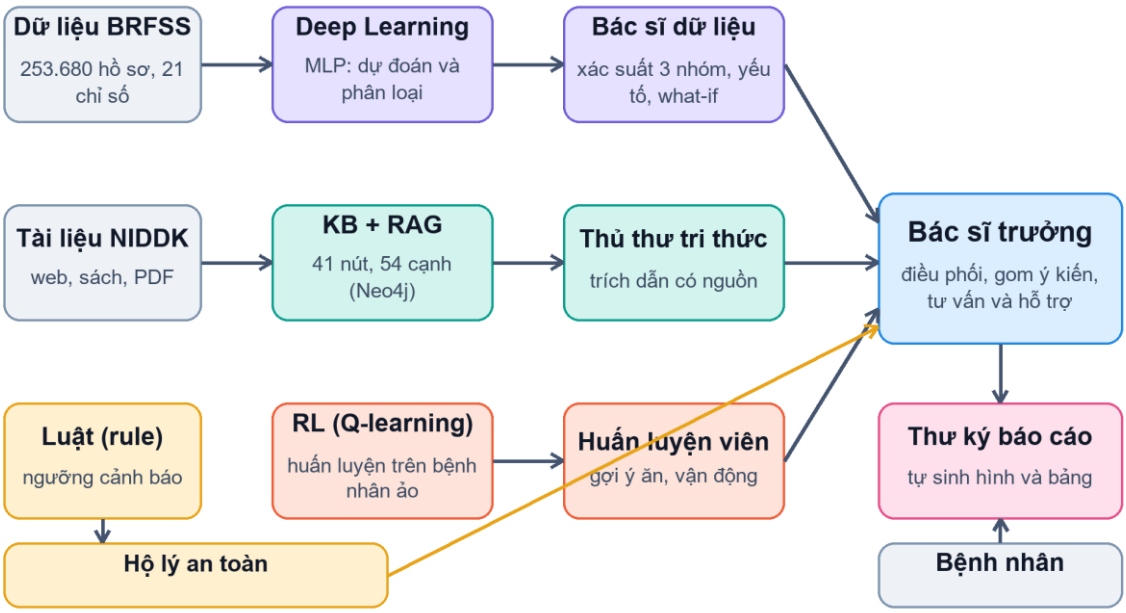
Bảng 2. So sánh một agent và hệ đa agent.

Tiêu chí	Một agent	Hệ đa agent (project này)
Độ phức tạp mỗi phần	Cao, trộn nhiều kỹ thuật	Thấp, mỗi agent một kỹ thuật
Kiểm thử	Khó tách lỗi	Kiểm thử từng agent, có 8 kiểm thử tự động
An toàn	Luật nằm lẫn với mô hình	Hộ lý an toàn độc lập và luôn được ưu tiên
Chi phí phối hợp	Không có	Cần bộ điều phối và giao thức sự kiện
Thay thế thành phần	Phải sửa chung	Thay từng agent (ví dụ đổi MLP thành mô hình khác)

Project dùng mô hình điều phối – chuyên gia (orchestrator–worker): Bác sĩ trưởng mời lần lượt từng agent, nhận kết quả có cấu trúc, rồi tổng hợp. Các agent không gọi trực tiếp nhau, nên luồng hội chẩn luôn có thể truy vết từng bước.

3. Kiến trúc tổng quan

Sơ đồ ở Hình 4 hiện thực hóa ý tưởng của đề bài: dữ liệu có sẵn đi qua Deep Learning để dự đoán và phân loại; tri thức từ web, sách, PDF được đưa vào Knowledge Base và truy xuất bằng RAG; agent huấn luyện viên kết hợp reinforcement learning với luật; tất cả hội tụ tại bộ điều phối để tư vấn và hỗ trợ bệnh nhân.



Hình 3. Kiến trúc tổng quan của hệ đa agent tiểu đường.

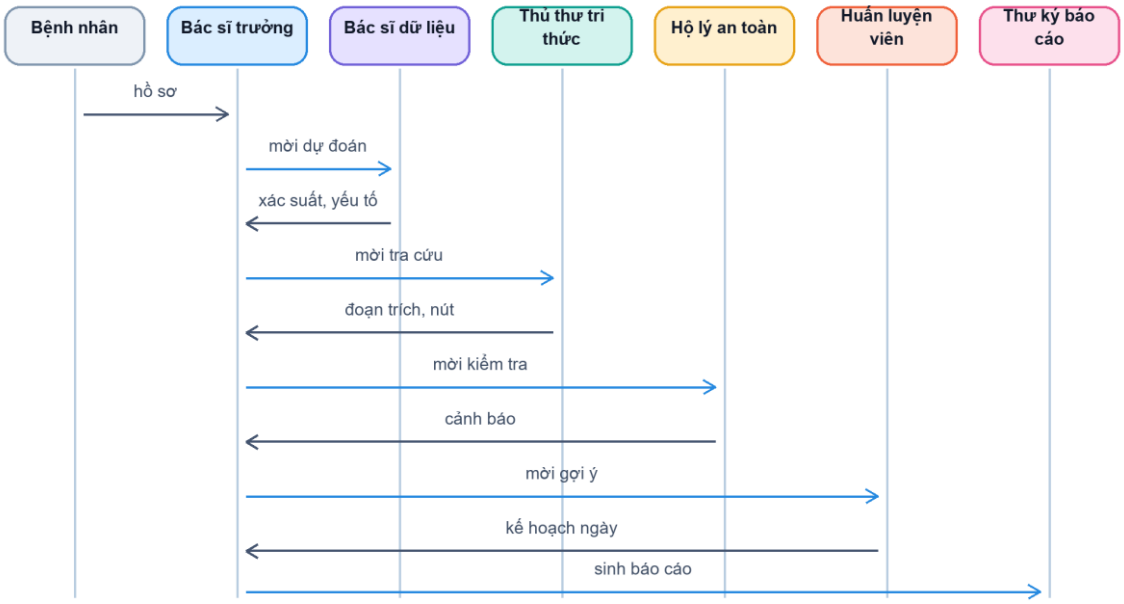
3.1. Vai trò của các agent

Bảng 3. Bảng phân công của sáu agent.

Agent	Kỹ thuật	Đầu vào	Đầu ra
Bác sĩ trưởng	Điều phối theo kịch bản	Hồ sơ bệnh nhân	Tóm tắt, mức nguy cơ, lời khuyên
Bác sĩ dữ liệu	Deep Learning (MLP)	21 chỉ số BRFSS suy ra từ hồ sơ	Xác suất 3 nhóm, yếu tố tác động, what-if
Thủ thư tri thức	Đồ thị tri thức + RAG	Hồ sơ, triệu chứng, câu hỏi	Nút liên quan, đoạn trích NIDDK có nguồn
Huấn luyện viên	Reinforcement learning	Bệnh nhân ảo suy từ hồ sơ	Kế hoạch ăn uống, vận động trong ngày
Hộ lý an toàn	Luật cố định	Hồ sơ, đường huyết, triệu chứng	Cảnh báo khẩn, lưu ý, nhắc nhở
Thư ký báo cáo	Sinh hình và bảng (matplotlib)	Kết quả của các agent	3 hình và 7 bảng báo cáo

3.2. Luồng một lượt hội chẩn

Mỗi lượt hội chẩn là một chuỗi tin nhắn có thứ tự (Hình 5). Máy chủ chạy toàn bộ chuỗi rồi trả về một danh sách sự kiện, giao diện 3D diễn hoạt từng sự kiện: agent nào nói, nói gì, truyền lượt cho ai, và nút nào của đồ thị tri thức được dùng.



Hình 4. Trình tự tin nhắn giữa các agent trong một lượt hội chẩn.

Bảng 4. Giao thức sự kiện giữa máy chủ và giao diện.

Loại sự kiện	Ý nghĩa	Ví dụ trên giao diện
speak	Một agent phát biểu	Bong bóng thoại, camera bay tới nhân vật
handoff	Bác sĩ trưởng chuyển lượt cho agent khác	Tia sáng nối hai nhân vật
tool	Agent chạy công cụ (mô hình, vẽ hình)	Màn hình của nhân vật sáng lên
cite	Thủ thư tri thức trích dẫn	Các nút trên bảng đồ thị phát sáng, hiện đoạn trích
alert	Hộ lý an toàn cảnh báo (info, warn, danger)	Đèn báo nhấp nháy theo màu mức độ
final	Kết luận của Bác sĩ trưởng	Bảng kết quả, hình và bảng báo cáo

3.3. Công nghệ và triển khai

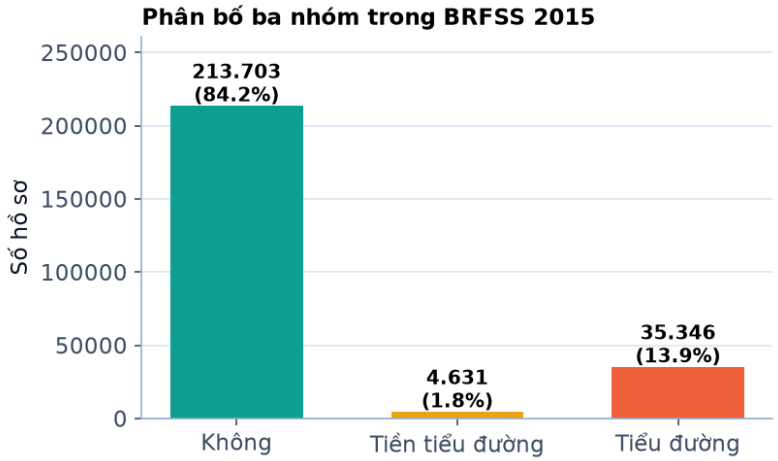
- **Máy chủ:** Python (thư viện chuẩn http.server), numpy cho suy luận, matplotlib cho hình. Chạy bằng một lệnh, cổng 8200, chỉ trên localhost.
- **Mô hình:** huấn luyện bằng PyTorch rồi xuất trọng số ra tệp npz; khi chạy demo chỉ cần numpy.
- **Giao diện:** Three.js (r128) dựng phòng khám 3D low-poly, JavaScript thuần, không cần npm hay kết nối mạng.
- **Neo4j:** đồ thị tri thức có thể nạp vào Neo4j bằng tệp Cypher đã xuất; khi không bật Neo4j, demo dùng bản đồ thị trong bộ nhớ.

4. Các thành phần của hệ thống

4.1. Deep Learning ⇒Agent

4.1.1. Dữ liệu và bài toán

Dữ liệu là bộ chỉ số sức khỏe tổng hợp từ khảo sát BRFSS 2015 của Trung tâm Kiểm soát và Phòng ngừa Bệnh tật Hoa Kỳ (CDC) [8]: 253.680 hồ sơ, 21 đặc trưng gồm huyết áp, mỡ máu, BMI, hút thuốc, vận động, tự đánh giá sức khỏe, tuổi theo 13 nhóm, học vấn và thu nhập. Nhãn có ba mức: không tiểu đường, tiền tiểu đường và tiểu đường. Tỉ lệ rất lệch: nhóm tiền tiểu đường chỉ có 4.631 hồ sơ (1,8%), nên đây là bài toán mất cân bằng lớp.



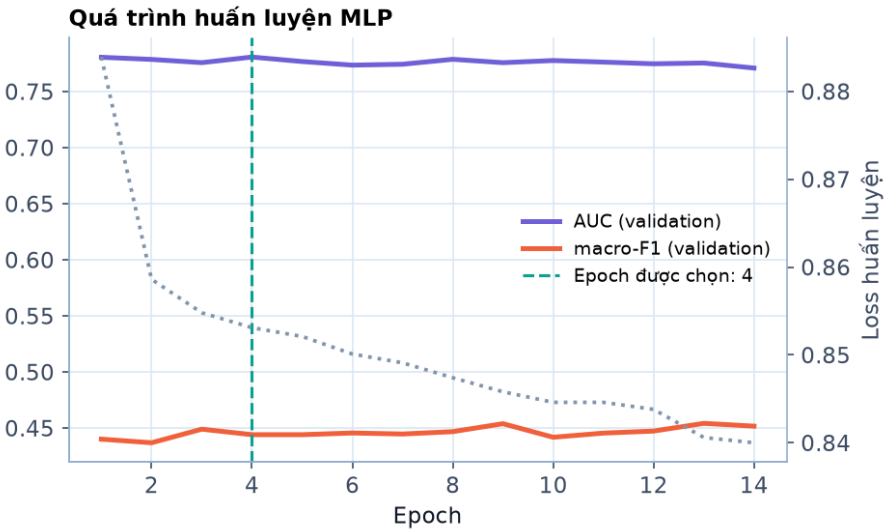
Hình 5. Phân bố ba nhóm trong dữ liệu.



Hình 6. Từ hồ sơ bệnh nhân đến lời giải thích của agent Deep Learning.

4.1.2. Mô hình và huấn luyện

Mô hình là mạng nơ-ron nhiều lớp (MLP) 21 -> 128 -> 64 -> 32 -> 3 (ReLU, dropout 0,15/0,1). Dữ liệu chia 80/20 theo phân tầng (202.944 huấn luyện, 50.736 kiểm tra); trong tập huấn luyện giữ lại 30.442 hồ sơ làm tập validation để chọn epoch và chỉnh ngưỡng. Hàm mất mát cross-entropy có trọng số lớp (mũ 0,75), tối ưu Adam, 14 epoch, batch 1.024; epoch tốt nhất theo AUC validation là epoch 4. Vì lớp tiền tiểu đường hiếm, nhãn dự đoán dùng hệ số điều chỉnh ngưỡng chọn trên validation (1,0, 1,5, 0,9); xác suất hiển thị cho người dùng vẫn là xác suất gốc của mô hình.



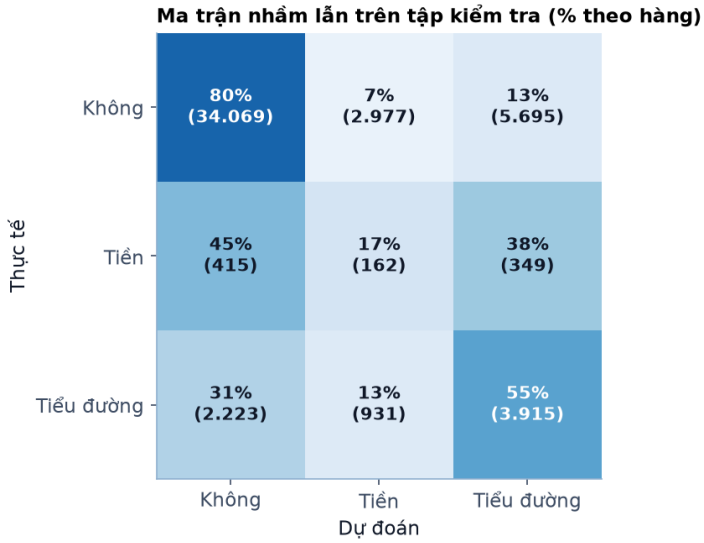
Hình 7. Đường học của MLP trên tập validation.

4.1.3. Kết quả

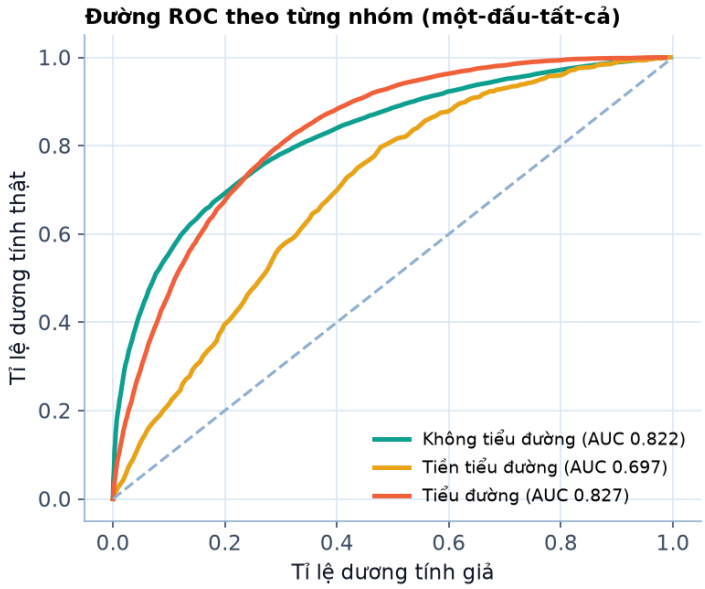
Bảng 5. Kết quả trên tập kiểm tra 50.736 hồ sơ.

Mô hình	Accuracy	Macro-F1	AUC (một-đấu-tất-cả)
Logistic Regression (cân bằng lớp)	64,6%	0,427	0,773
MLP, lấy lớp xác suất cao nhất	78,0%	0,445	0,782
MLP, có hệ số điều chỉnh ngưỡng (dùng trong demo)	75,2%	0,461	0,782

Độ nhạy (recall) theo nhóm khi dùng hệ số ngưỡng: không 80%, tiền tiểu đường 17%, tiểu đường 55%. Nhận xét trung thực: MLP chỉ hơn hồi quy logistic một chút về AUC, và nhóm tiền tiểu đường gần như không phân biệt được bằng các chỉ số khảo sát này. Vì vậy hệ thống dùng chỉ số nguy cơ kết hợp (xác suất tiểu đường cộng một nửa xác suất tiền tiểu đường) và luôn khuyên xét nghiệm xác nhận, thay vì coi nhãn dự đoán là kết luận.



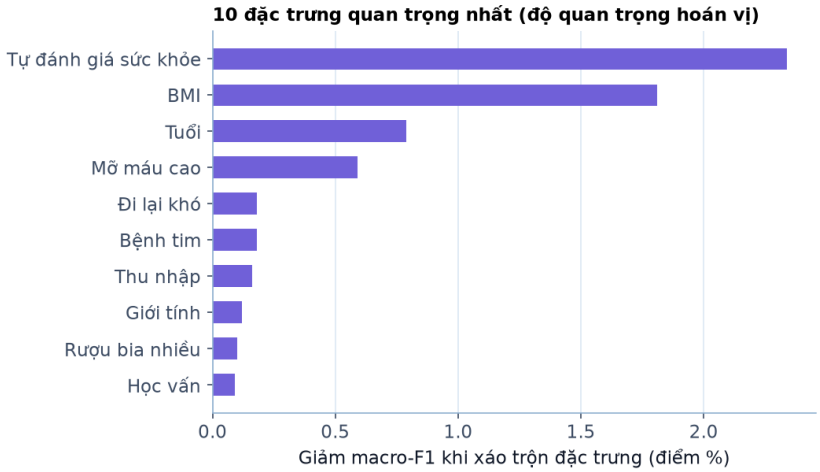
Hình 8. Ma trận nhầm lẫn trên tập kiểm tra.



Hình 9. Đường ROC theo từng nhóm.

4.1.4. Từ mô hình thành agent

Một con số xác suất chưa đủ để tư vấn. Agent Deep Learning trả thêm hai phần giải thích. Thứ nhất, tác động của từng yếu tố: thay giá trị của yếu tố đó bằng trung bình dân số rồi đo chỉ số nguy cơ thay đổi bao nhiêu. Thứ hai, what-if: nếu bệnh nhân thay đổi được một yếu tố lối sống (BMI về 24,5, vận động, bỏ thuốc, hạn chế rượu bia, ăn rau và trái cây) thì chỉ số nguy cơ giảm bao nhiêu. Hai phần này là thông tin mô hình học được từ dữ liệu quan sát, không phải bằng chứng nhân quả.

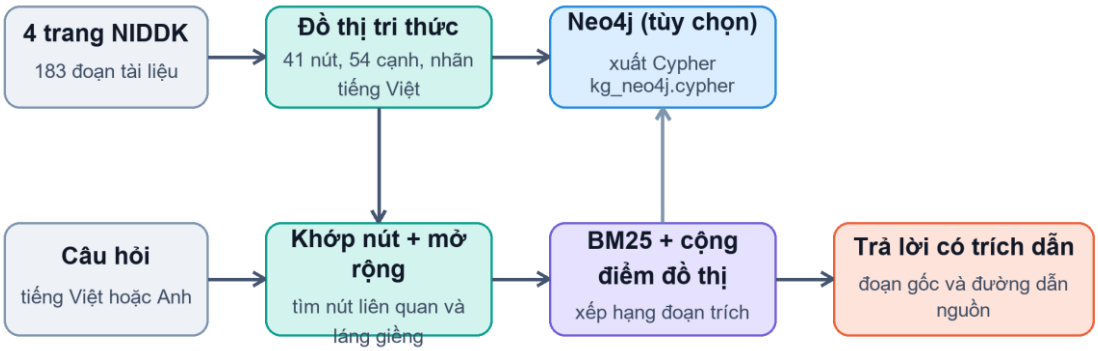


Hình 10. Độ quan trọng hoán vị của 10 đặc trưng.

4.2. Knowledge Base và RAG

4.2.1. Nguồn tri thức và đồ thị

Nguồn tri thức là bốn trang của NIDDK (what-is-diabetes, symptoms-causes, tests-diagnosis, managing-diabetes) được thu thập thành 183 đoạn văn [4]. Nhóm biên soạn đồ thị tri thức gồm 41 nút và 54 cạnh với nhãn tiếng Việt; mỗi nút gắn với các đoạn trích liên quan nhất bằng BM25. Các ngưỡng xét nghiệm (A1C, đường huyết đói, OGTT) theo chuẩn của Hiệp hội Đái tháo đường Hoa Kỳ (ADA) [9] được ghi là tri thức bổ sung, vì bảng số trong trang NIDDK bị mất khi thu thập tài liệu.



Hình 11. Xây dựng đồ thị tri thức và luồng truy xuất RAG.

Bảng 6. Các loại nút trong đồ thị tri thức.

Loại nút	Số nút	Ví dụ
Bệnh/thể bệnh	6	Bệnh tiểu đường, Tiểu đường típ 1, Tiểu đường típ 2, Tiền tiểu đường, Tiểu đường thai kỳ, …
Triệu chứng	6	Khát nước nhiều, Đi tiểu nhiều, Mệt mỏi, Nhìn mờ, Vết thương lâu lành, Sụt cân không rõ lý…
Biến chứng	8	Nhiễm toan ceton, Hạ đường huyết, Tăng đường huyết, Bệnh tim mạch, Bệnh thận, Tổn thương m…

Yếu tố nguy cơ	8	Thừa cân, béo phì, Ít vận động, Tuổi từ 35 trở lên, Tiền sử gia đình, Huyết áp cao, Mỡ máu…
Xét nghiệm	5	Xét nghiệm A1C, Đường huyết lúc đói (FPG), Nghiệm pháp dung nạp glucose (OGTT), Đường huyế…
Lối sống	4	Giữ cân nặng hợp lý, Vận động đều đặn, Ăn uống lành mạnh, Bỏ thuốc lá
Điều trị, theo dõi	4	Tự theo dõi đường huyết, Thời gian trong ngưỡng an toàn, Thuốc điều trị tiểu đường, Điều t…

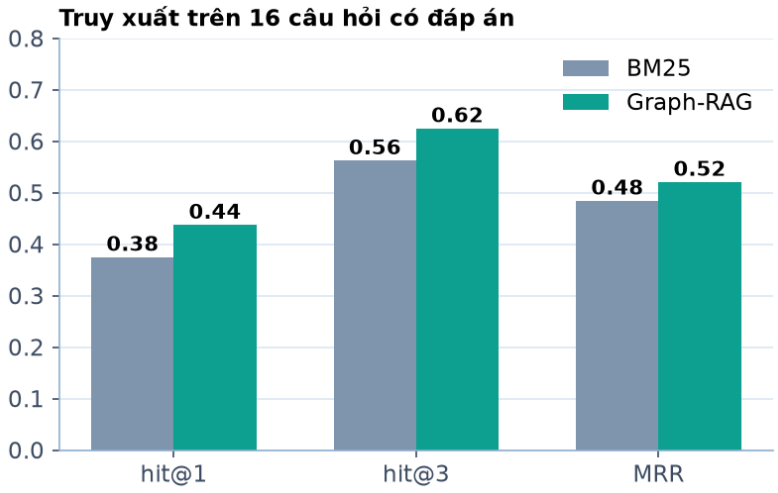
Đồ thị được xuất ra tệp Cypher (kg_neo4j.cypher) để nạp vào Neo4j: mỗi nút là một nhãn KBNode với thuộc tính id, vi, type, desc; mỗi quan hệ như RAISES_RISK, DIAGNOSED_BY, SYMPTOM_OF là một cạnh có hướng. Phòng khám 3D vẽ đồ thị này thành bảng tri thức trong phòng và thành bản đồ tri thức 3D.

4.2.2. Graph-RAG và đánh giá

Graph-RAG của project chạy ba bước: tìm các nút khớp câu hỏi (theo tên tiếng Việt, tiếng Anh và từ đồng nghĩa), mở rộng một bước sang các nút láng giềng, rồi dùng BM25 trên câu hỏi và cộng điểm cho các đoạn trích gắn với những nút đó (nút khớp +1,0; nút láng giềng +0,3). Kết quả trả về là đoạn trích nguyên văn kèm đường dẫn nguồn, không sinh văn bản tự do, nên không có nguy cơ bịa nội dung y khoa.

Bảng 7. Truy xuất trên 16 câu hỏi có đáp án (data/chapter04/diabetes_qa_demo.json).

Phương pháp	**hit@1**	**hit@3**	**MRR**
BM25 thuần	0,38	0,56	0,48
Graph-RAG (BM25 + cộng điểm đồ thị)	0,44	0,62	0,52



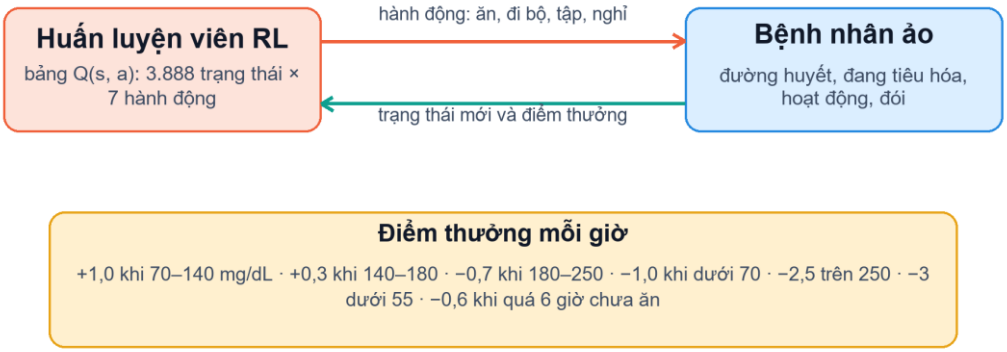
Hình 12. So sánh BM25 và Graph-RAG.

Hạn chế. Graph-RAG cải thiện nhẹ so với BM25 thuần. Cần thận trọng khi diễn giải: bộ đánh giá chỉ có 16 câu tiếng Anh, trọng số cộng điểm được chọn thử trên chính bộ này, nên kết quả chỉ cho thấy xu hướng. Hạn chế khác: tài liệu gốc bằng tiếng Anh nên đoạn trích hiển thị nguyên văn tiếng Anh, còn nhãn nút và lời dẫn bằng tiếng Việt.

4.3. Reinforcement Learning: huấn luyện viên lối sống

4.3.1. Môi trường mô phỏng

Vì không thể thử nghiệm lên người thật, nhóm xây dựng một bệnh nhân ảo đơn giản. Mỗi bước là một giờ, một ngày có 24 bước từ 6 giờ sáng đến 5 giờ sáng hôm sau. Đường huyết tăng sau bữa ăn (nhanh hơn với đồ ngọt, mạnh hơn khi kháng insulin), giảm khi vận động và khi dùng thuốc, và tự kéo dần về mức nền. Ban đêm bệnh nhân chỉ được nghỉ hoặc ăn nhẹ có đường. Các tham số được đặt theo trực giác, không hiệu chỉnh từ số liệu bệnh nhân thật.



Hình 13. Vòng lặp tương tác giữa huấn luyện viên RL và bệnh nhân ảo.

Bảng 8. Thiết kế bài toán RL.

Thành phần	Mô tả
Trạng thái	Khối giờ trong ngày (6), nhóm đường huyết (6), lượng tinh bột đang tiêu hóa (3), hoạt động gần đây (2), mức đói (3), độ nhạy insulin (3), có dùng thuốc (2): 3.888 trạng thái
Hành động	7 hành động: nghỉ, ăn nhẹ có đường, bữa nhẹ, bữa chính, bữa nhiều tinh bột, đi bộ, tập thể dục
Phần thưởng	Theo vùng đường huyết (+1,0 khi 70–140 mg/dL ... −3 khi dưới 55), phạt −0,6 khi quá 6 giờ chưa ăn, thưởng +0,25 cho hai lần vận động đầu tiên trong ngày
Thuật toán	Q-learning dạng bảng, alpha 0,15, gamma 0,97, 60.000 ngày huấn luyện trên bệnh nhân ngẫu nhiên, epsilon giảm dần

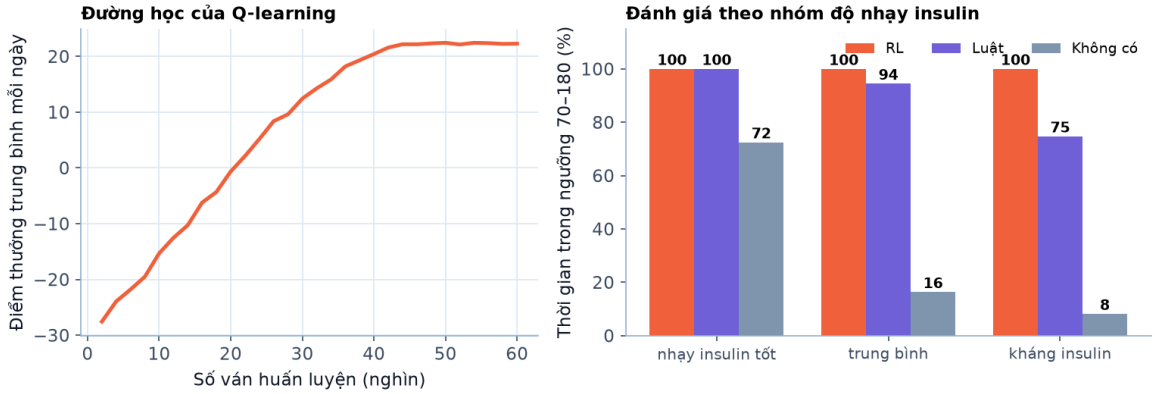
4.3.2. Kết quả so với luật và không có huấn luyện viên

Huấn luyện mất 93,2 giây, đã gặp 1.535 trạng thái. Đánh giá trên 70 bệnh nhân ngẫu nhiên mới mỗi nhóm, mỗi người 5 ngày, với ba chính sách: huấn luyện viên RL, huấn luyện viên theo luật (ăn đúng giờ, ăn nhẹ khi thấp, đi bộ khi cao) và người không có huấn luyện viên (ăn đúng giờ, ăn vặt đồ ngọt, ít vận động).

Bảng 9. Đánh giá huấn luyện viên theo nhóm độ nhạy insulin.

Nhóm bệnh nhân	Chính sách	Trong ngưỡng 70–180	Đường huyết TB	Giờ thấp	Giờ cao	Điểm/ngày
nhạy insulin tốt (s ≥ 0,85)	Huấn luyện viên RL	100%	110	0,0	0,0	23,0

	Huấn luyện viên theo luật	100%	133	0,0	0,0	17,2
	Không có huấn luyện viên	72%	162	0,0	6,6	3,5
trung bình (0,55 ≤ s < 0,85)	Huấn luyện viên RL	100%	108	0,0	0,0	23,4
	Huấn luyện viên theo luật	94%	152	0,0	1,3	10,1
	Không có huấn luyện viên	16%	231	0,0	20,1	-28,6
kháng insulin (s < 0,55)	Huấn luyện viên RL	100%	113	0,0	0,0	23,2
	Huấn luyện viên theo luật	75%	160	0,0	6,1	4,8
	Không có huấn luyện viên	8%	282	0,0	22,1	-43,7



Hình 14. Đường học của Q-learning và so sánh ba chính sách.

Diễn giải. Huấn luyện viên RL giữ gần như toàn bộ thời gian trong ngưỡng an toàn, luật làm tốt với bệnh nhân nhạy insulin nhưng kém dần khi kháng insulin tăng, còn người không có huấn luyện viên chỉ đạt rất thấp ở nhóm kháng insulin. Con số gần 100% của RL phản ánh việc môi trường mô phỏng khá đơn giản và có thể học thuộc, chứ không có nghĩa agent sẽ đạt kết quả như vậy với người thật. Đây là điểm cần nói rõ khi trình bày.

4.4. Agent luật an toàn

Hộ lý an toàn là agent dựa trên luật (rule-based): quy tắc cố định do con người viết, không học. Lý do dùng luật cho phần này là cần hành vi dự đoán được và giải thích được khi liên quan đến an toàn. Agent chạy độc lập với mô hình, và nếu có luật mức khẩn thì Bác sĩ trưởng luôn nâng mức nguy cơ lên cao.

Bảng 10. Mười luật của Hộ lý an toàn.

Luật	Mức	Điều kiện	Căn cứ
R1	Khẩn	Đường huyết ≥ 250 mg/dL	Ngưỡng cảnh báo của demo; đường huyết ngẫu nhiên ≥ 200 kèm triệu chứng (ADA/NIDDK)
R2	Khẩn	≥ 200 mg/dL kèm triệu chứng điển hình	ADA/NIDDK
R3	Lưu ý	Đường huyết đói ≥ 126 mg/dL	Chuẩn ADA, cần lặp lại xét nghiệm

R4	Lưu ý	Đường huyết đói 100–125 mg/dL	Chuẩn ADA, vùng tiền tiểu đường
R5	Khẩn	Đường huyết < 70 mg/dL	NIDDK: dưới 70 mg/dL là quá thấp
R6	Lưu ý	≥ 3 triệu chứng điển hình	NIDDK: có triệu chứng nên xét nghiệm
R7	Lưu ý	BMI ≥ 30	NIDDK: thừa cân, béo phì là yếu tố nguy cơ
R8	Lưu ý	Huyết áp cao, mỡ máu cao và bệnh tim hoặc đột quỵ	NIDDK: biến chứng tim mạch
R9	Nhắc	Từ 45 tuổi chưa đo mỡ máu	NIDDK: tầm soát định kỳ
R10	Lưu ý	Mô hình ước lượng tiểu đường ≥ 50%	Quy tắc của demo: khuyên xét nghiệm xác nhận

4.5. Bác sĩ trưởng và báo cáo tự sinh

Bác sĩ trưởng chạy các agent theo thứ tự cố định, nhưng quyết định mức nguy cơ bằng quy tắc kết hợp: dựa trên chỉ số nguy cơ của mô hình (dưới 10% là thấp, dưới 25% là trung bình, từ 25% là cao) và nâng mức khi có cảnh báo của Hộ lý an toàn. Lời khuyên cuối cùng gồm cảnh báo ưu tiên, các thay đổi lối sống hiệu quả nhất theo what-if, kế hoạch ngày của huấn luyện viên và lời nhắc xét nghiệm xác nhận.

Thư ký báo cáo tự sinh cho mỗi lượt khám ba hình (xác suất ba nhóm, yếu tố tác động và what-if, một ngày mô phỏng của ba kiểu huấn luyện) và bảy bảng (hồ sơ so với dân số, nguy cơ, what-if, cảnh báo, kế hoạch ngày, so sánh huấn luyện viên, nguồn trích dẫn). Người dùng có thể tải báo cáo này thành một tệp HTML tự chứa ảnh.

5. Code và Demo

5.1. Cấu trúc mã nguồn

```
project01/
├── server.py             máy chủ localhost (API + tệp tĩnh), cổng 8200
├── mas/                  các agent
│   ├── data.py           hồ sơ bệnh nhân, ánh xạ sang 21 chỉ số BRFSS, vai mẫu
│   ├── dl_agent.py       Deep Learning: suy luận MLP, yếu tố tác động, what-if
│   ├── kb.py             đồ thị tri thức, BM25, Graph-RAG
│   ├── rl.py             mô phỏng 24 giờ, Q-learning, huấn luyện viên theo luật
│   ├── rules.py          Hộ lý an toàn (10 luật)
│   ├── orchestrator.py   Bác sĩ trưởng: kịch bản hội chẩn, kết luận
│   └── report.py         Thư ký báo cáo: hình và bảng
├── train/                train_dl.py, build_kg.py, train_rl.py, eval_rag.py
├── artifacts/            dl_model.npz, kg.json, kg_neo4j.cypher, rl_qtable.json, *_metrics.json
└── static/               giao diện 3D (index.html, scene.js, app.js, style.css, vendor/three)
```

5.2. Các hàm chính

Bảng 11. Các hàm chính của hệ thống.

Hàm	Vai trò
DLAgent.predict	Ánh xạ hồ sơ sang vector 21 chiều, chạy MLP, tính yếu tố tác động và what-if
KnowledgeBase.search	Graph-RAG: khớp nút, mở rộng láng giềng, BM25 có cộng điểm đồ thị, trả đoạn trích kèm nguồn
rl.step	Một giờ mô phỏng: hấp thu tinh bột, tác dụng vận động và thuốc, nhiễu, phần thưởng
rl.train	Vòng huấn luyện Q-learning epsilon-greedy trên bệnh nhân ngẫu nhiên
rules.evaluate	Duyệt 10 luật an toàn và trả về các luật được kích hoạt kèm căn cứ
Clinic.consult	Chạy tuần tự các agent, dựng danh sách sự kiện, mức nguy cơ, lời khuyên và báo cáo

Agent Deep Learning: hàm predict ánh xạ hồ sơ, chạy mô hình rồi tính giải thích.

```
def predict(self, profile: dict) -> dict:
    prof = normalize(profile)
    x = np.array([to_vector(prof)], dtype="float32")
    p = self._forward(x)[0]
    label = CLASSES[int((p * self.scale).argmax())]
    risk_index = float(self.index(p[None])[0])
    # tác động của từng yếu tố: thay bằng giá trị trung bình dân số
    base = float(self.index(self._forward(x))[0])
    drivers = []
    for j, name in enumerate(FEATURES):
        if name in ("CholCheck", "AnyHealthcare", "Education", "Income", "NoDocbcCost"):
            continue
        x2 = x.copy()
        x2[0, j] = self.pop_mean[j]
        delta = base - float(self.index(self._forward(x2))[0])
        drivers.append(dict(feature=name, vi=VI_FEATURE[name], value=float(x[0, j]),
effect=round(delta, 4)))
    drivers.sort(key=lambda d: -abs(d["effect"]))
    what_if = []
    for name, healthy, text in MODIFIABLE:
        j = FEATURES.index(name)
        if (name == "BMI" and x[0, j] <= healthy + 0.5) or (name != "BMI" and x[0, j] == healthy):
            continue
        x2 = x.copy()
```

```
        x2[0, j] = healthy
        gain = base - float(self.index(self._forward(x2))[0])
        what_if.append(dict(feature=name, action=text, from_value=float(x[0, j]),
to_value=healthy, gain=round(gain, 4)))
    what_if.sort(key=lambda d: -d["gain"])
    return dict(risk=dict(none=float(p[0]), pre=float(p[1]), diabetes=float(p[2])), label=label,
risk_index=risk_index,
                bmi=float(x[0, 3]), drivers=drivers[:6], what_if=what_if)
```

Đoạn mã 1. DLAgent.predict (rút gọn docstring).

Agent tri thức: Graph-RAG khớp nút, mở rộng và cộng điểm cho đoạn trích gắn với nút.

```
def search(self, query: str, k: int = 3, boost: dict[str, float] | None = None) -> list[dict]:
    sc = self.scores(tokens(query))
    if boost:
        sc = [s + boost.get(d["id"], 0.0) for s, d in zip(sc, self.docs)]
    # bỏ các đoạn bị cụt (bắt đầu bằng chữ thường, thường là mục gạch đầu dòng) khi chọn đoạn để
trích dẫn
    order = [i for i in sorted(range(len(sc)), key=lambda i: -sc[i]) if not
self.docs[i]["text"][0].islower()][:k]
    return [dict(id=self.docs[i]["id"], quote=self.docs[i]["text"], url=self.docs[i]["url"],
score=round(sc[i], 3)) for i in order if sc[i] > 0]
```

Đoạn mã 2. KnowledgeBase.search.

Môi trường mô phỏng và vòng huấn luyện Q-learning của huấn luyện viên.

```
def step(state: dict, action: str, rng: random.Random) -> tuple[dict, float, dict]:
    p = state["patient"]
    s, base, med = p["insulin_sensitivity"], p["baseline_glucose"], p.get("medication", 0.0)
    act = ACTIONS[AID[action]]
    g, c, a = state["glucose"], state["carbs"], state["act"]
    c_total = c + act["carbs"]
    absorbed = c_total * (0.75 if act["fast"] else 0.45)
    c_after = c_total - absorbed
    rise = absorbed * 1.35 * (1.7 - s)
    a_total = a + act["act"]
    drop_act = a_total * 14 * (0.5 + 0.5 * s) * max(0.0, min(1.0, (g - 62) / 60))
    pull = (g - base) * 0.16 * s
    drop_med = med * max(0.0, min(1.0, (g - 55) / 60))
    g2 = g + rise - drop_act - pull - drop_med + rng.gauss(0, 3)
    g2 = max(35.0, min(420.0, g2))
    meal = action in MEALS
    since = 0 if meal else state["since_meal"] + 1
    reward = zone_reward(g2)
    night = is_night(state["hour"] + 1)
    if since >= 6 and not night:
        reward -= 0.6
    ex = state["exercise"] + (1 if act["act"] > 0 else 0)
    if act["act"] > 0 and state["exercise"] < 2:
        reward += 0.25
    new = dict(hour=state["hour"] + 1, glucose=round(g2, 1), carbs=round(c_after, 2),
act=round(a_total * 0.5, 3), since_meal=since,
               exercise=ex, patient=p, log=state["log"] + [dict(hour=state["hour"], action=action,
glucose=round(g2, 1))])
    return new, reward, dict(zone=zone(g2), delta=round(g2 - g, 1), rise=round(rise, 1),
drop=round(drop_act + drop_med, 1))
```

Đoạn mã 3. rl.step: một giờ mô phỏng.

```
def train(episodes: int = 60000, seed: int = 11, alpha: float = 0.15, gamma: float = 0.97,
log_every: int = 2000):
    rng = random.Random(seed)
    coach = QCoach()
    curve = []
    window = []
    for ep in range(episodes):
        eps = max(0.05, 1.0 - 0.95 * ep / (0.7 * episodes))
        st = new_state(random_patient(rng), rng)
        total = 0.0
        while not done(st):
            key = encode(st)
            a = coach.act(st, eps, rng)
            nst, r, _ = step(st, a, rng)
            fin = done(nst)
            target = r if fin else r + gamma * max(coach.values(encode(nst))[AID[x]] for x in
allowed(nst["hour"]))
            v = coach.values(key)
            v[AID[a]] += alpha * (target - v[AID[a]])
            st, total = nst, total + r
        window.append(total)
        if (ep + 1) % log_every == 0:
            curve.append(dict(episode=ep + 1, mean_return=round(sum(window) / len(window), 3),
epsilon=round(eps, 3)))
            window = []
    return coach, curve
```

Đoạn mã 4. rl.train: Q-learning.

Điều phối: mức nguy cơ kết hợp xác suất của mô hình với cảnh báo của luật.

```
def level(self, dl: dict, rule_list: list[dict]) -> str:
    r = dl["risk_index"]
    lv = "thấp" if r < 0.10 else "trung bình" if r < 0.25 else "cao"
    top = rules.top_level(rule_list)
    if top == "danger":
        return "cao"
    if top == "warn" and lv == "thấp":
        return "trung bình"
    return lv
```

Đoạn mã 5. Clinic.level.

5.3. Cách chạy

```
# từ thư mục gốc dự án
.\.venv\Scripts\python.exe project01/server.py --open     # mở http://localhost:8200

# huấn luyện lại (tùy chọn)
.\.venv\Scripts\python.exe project01/train/train_dl.py     # Deep Learning trên BRFSS
python project01/train/build_kg.py                              # đồ thị tri thức và Cypher
python project01/train/train_rl.py                              # Q-learning
python project01/train/eval_rag.py                              # đánh giá RAG

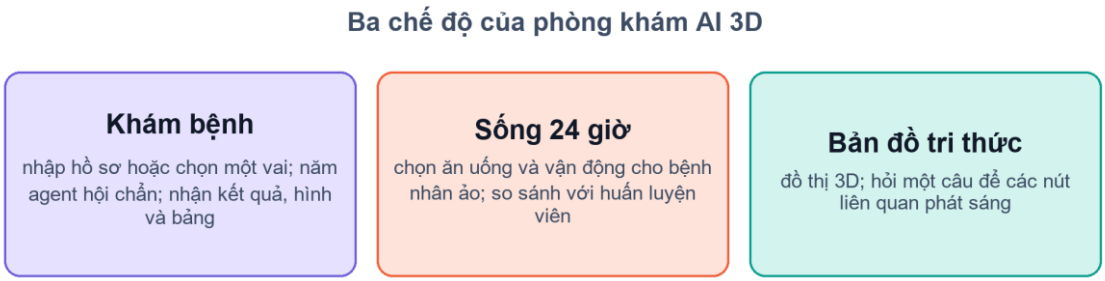
# notebook thực hành (Jupyter trên localhost)
.\.venv\Scripts\python.exe -m jupyter lab notebooks/project01

# kiểm thử
.\.venv\Scripts\python.exe -m pytest tests/test_project01.py -q
```

Đoạn mã 6. Lệnh chạy; toàn bộ lệnh của dự án nằm trong RUN.md.

5.4. Demo: phòng khám AI 3D

Giao diện là một phòng khám 3D low-poly. Bác sĩ trưởng đứng giữa phòng, năm agent còn lại ngồi ở các bàn riêng với màu và dáng khác nhau; bệnh nhân là nhân vật của chính người dùng. Người dùng có ba chế độ chơi như Hình 6.



Hình 15. Ba chế độ của demo.

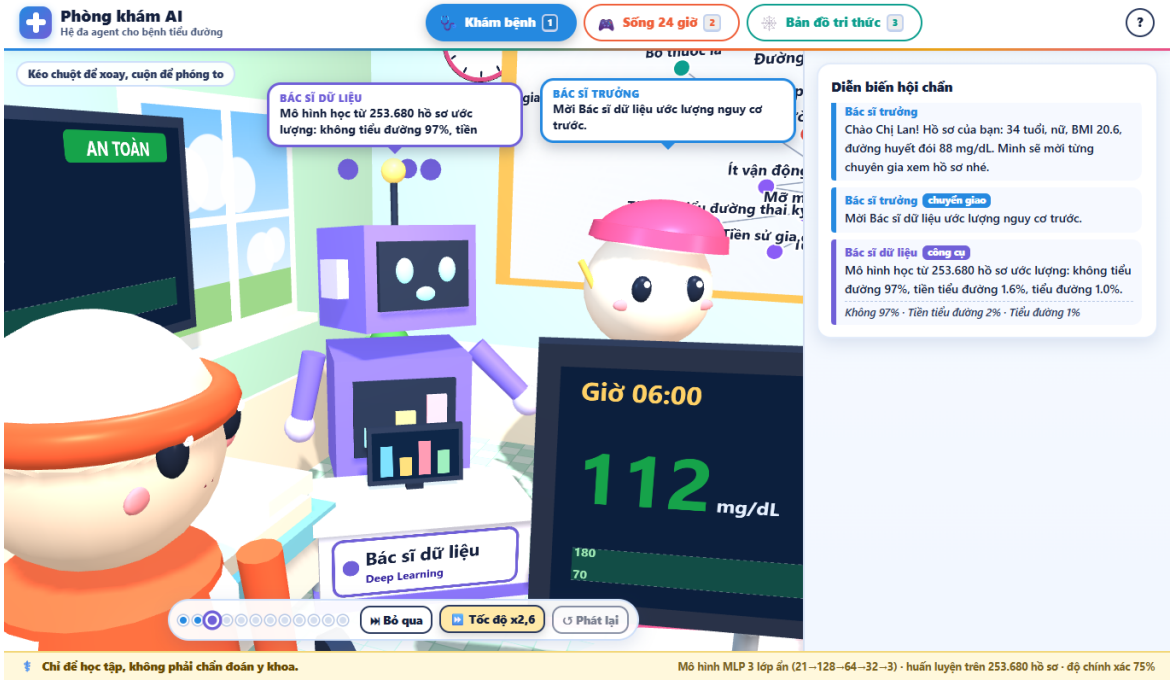
Mỗi câu hội thoại hiện thành phụ đề lớn ở dưới màn hình (bong bóng trên đầu nhân vật chỉ giữ phần rút gọn) và được đọc bằng giọng tiếng Việt qua Azure Speech khi có khóa trong tệp .env; chữ chạy theo đúng độ dài của giọng. Người dùng có thể tạm dừng (phím Dấu cách), tắt tiếng (phím V) hoặc tua nhanh. Khóa chỉ nằm ở máy chủ, trình duyệt chỉ nhận file âm thanh, và nếu không có khóa thì demo vẫn chạy với chữ.

Chế độ khám bệnh: người dùng chọn một vai có sẵn hoặc tự nhập hồ sơ rồi bước vào phòng khám.



Hình 16. Toàn cảnh phòng khám 3D với sáu agent và bệnh nhân.

Mỗi sự kiện được diễn hoạt: bong bóng thoại, tia sáng chuyển lượt, đèn cảnh báo theo mức độ.



Hình 17. Hội chẩn: các agent lần lượt phát biểu, camera bay tới người đang nói.

Kết quả gồm mức nguy cơ, lời khuyên theo thứ tự ưu tiên và các cảnh báo của Hộ lý an toàn.



Hình 18. Bảng kết quả: nguy cơ ba nhóm, mức độ, lời khuyên, cảnh báo và trích dẫn.

Báo cáo do Thư ký báo cáo vẽ ngay sau lượt khám, đúng yêu cầu sinh ra hình và bảng.



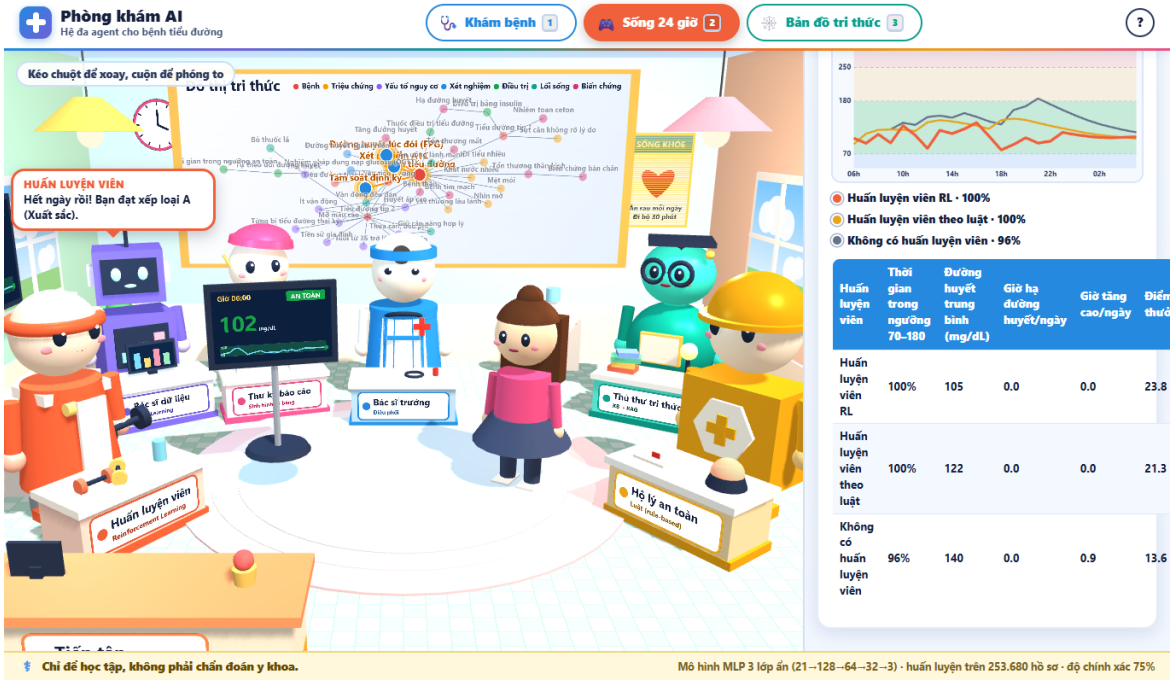
Hình 19. Báo cáo tự sinh với hình và bảng, có thể tải về dưới dạng HTML.

Màn hình theo dõi đường huyết 3D và gợi ý của huấn luyện viên RL và của luật xuất hiện ở mỗi lượt.



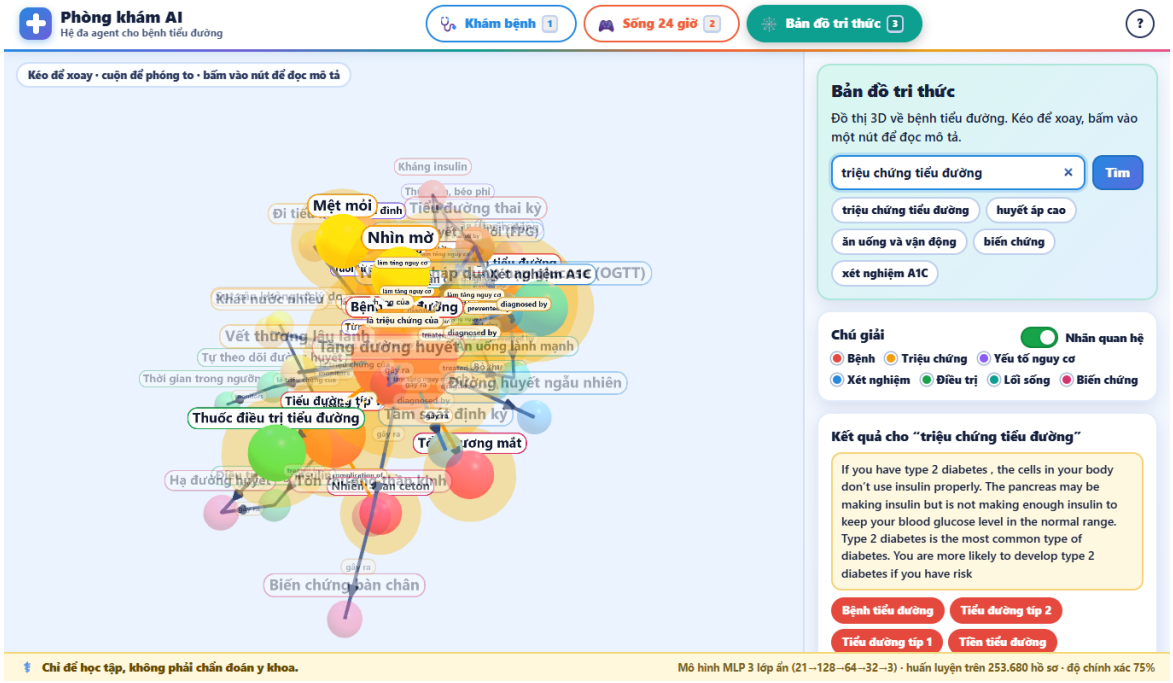
Hình 20. Sống 24 giờ: người dùng chọn ăn uống và vận động cho bệnh nhân ảo.

Đường cong 24 giờ và bảng số liệu của ba huấn luyện viên cho đúng bệnh nhân đang chơi.



Hình 21. So sánh huấn luyện viên RL, luật và không có huấn luyện viên.

Hỏi một câu thì các nút liên quan phát sáng, kèm đoạn trích RAG có nguồn.



Hình 22. Bản đồ tri thức 3D: nút phát sáng theo câu hỏi.

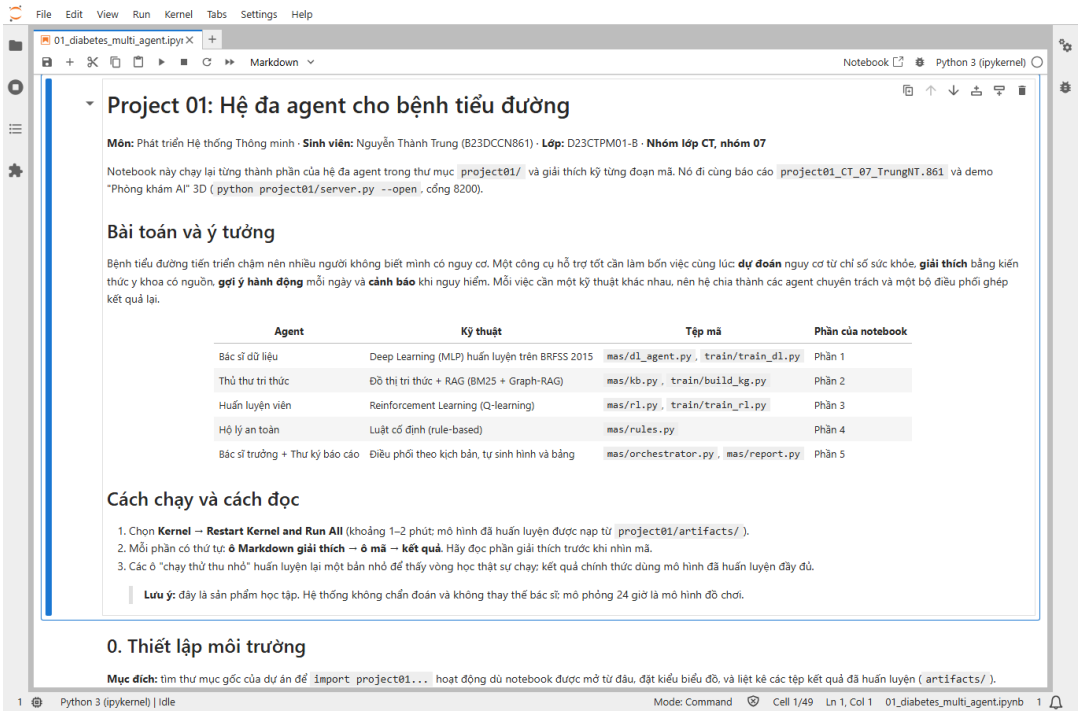
5.5. Notebook thực hành

Toàn bộ các thành phần được chạy lại từng bước trong notebook notebooks/project01/01_diabetes_multi_agent.ipynb (64 ô, gồm các phần 5b–5f bổ sung cho bản nâng cấp). Mỗi phần có thứ tự ô giải thích, ô mã, kết quả, và cuối notebook có tổng kết,

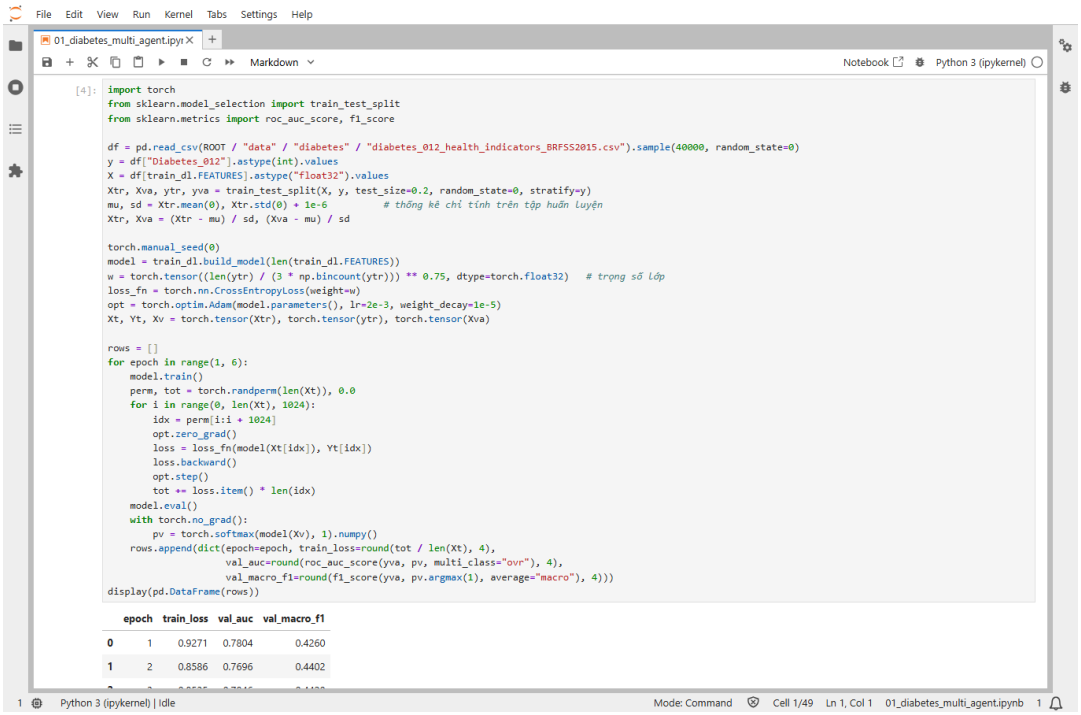
hạn chế cùng sáu câu hỏi trình bày. Notebook mở bằng Jupyter Lab trên localhost (python -m jupyter lab notebooks/project01) và đã được chạy sẵn nên có thể đọc kết quả ngay; chọn Kernel, Restart Kernel and Run All để chạy lại trong khoảng 1–2 phút.

Bảng 12. Cấu trúc của notebook Project 01.

Phần	Nội dung và ô tiêu biểu
0. Thiết lập	Tìm thư mục gốc, nạp thư viện, liệt kê tệp mô hình đã huấn luyện; hàm show_source in mã nguồn bằng inspect
1. Deep Learning	Phân bố ba nhóm, mã build_model và numpy_forward, chạy thử thu nhỏ vòng huấn luyện, đường học, ma trận nhầm lẫn, ROC, DLAgent.predict, yếu tố tác động và what-if
2. KB và RAG	Đếm nút và cạnh, vẽ đồ thị tri thức, mã KnowledgeBase.search, so BM25 với Graph-RAG, bảng hit@k và MRR
3. Reinforcement Learning	Bảng hành động và phần thưởng, mã rl.step, thử ba hành động, mã rl.train, huấn luyện thu nhỏ 4.000 ngày, so ba huấn luyện viên
4. Luật an toàn	Mã rules.evaluate và các luật được kích hoạt cho hai vai mẫu
5. Bác sĩ trưởng	Chạy Clinic.consult, bảng 13 sự kiện, tóm tắt, ba hình và ba bảng báo cáo tự sinh
6. Dịch vụ và kiểm thử	Mô tả API, chạy 8 kiểm thử tự động ngay trong notebook



Hình 23. Đầu notebook: mục tiêu, bảng agent theo kỹ thuật và tệp mã, cách chạy và cách đọc.



```
[4]: import torch
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, f1_score

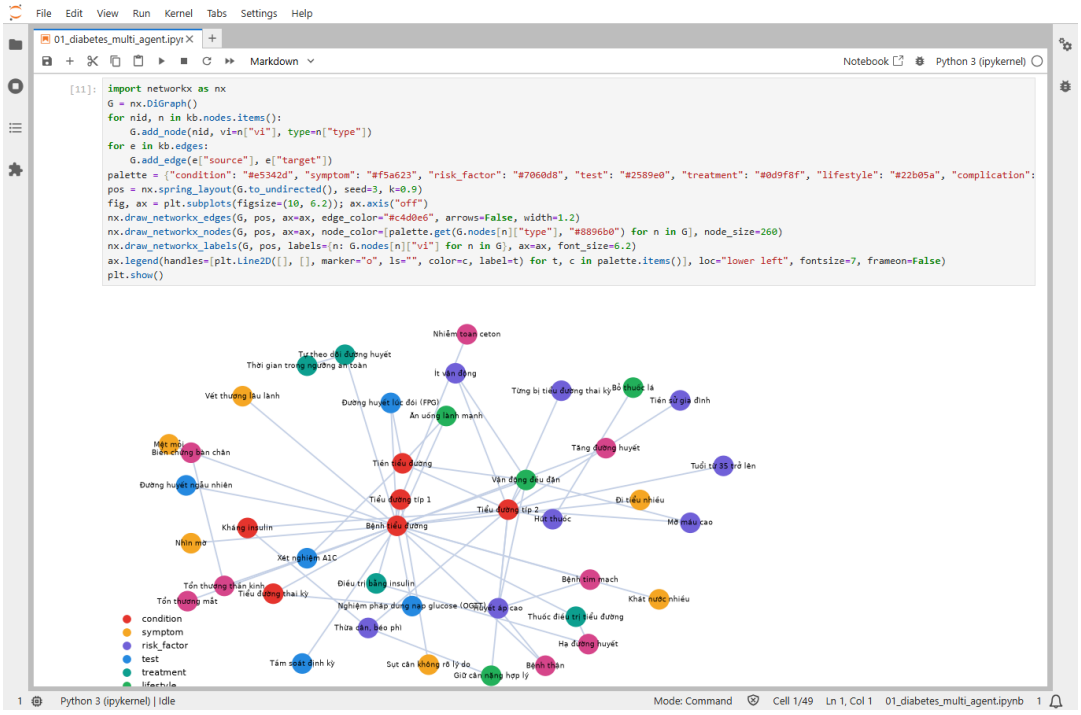
df = pd.read_csv(ROOT / "data" / "diabetes" / "diabetes_012_health_indicators_BRFSS2015.csv").sample(40000, random_state=0)
y = df["Diabetes_012"].astype(int).values
X = df[train_dl.FEATURES].astype("float32").values
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.2, random_state=0, stratify=y)
mu, sd = Xtr.mean(0), Xtr.std(0) + 1e-6          # thống kê chỉ tính trên tập huấn luyện
Xtr, Xva = (Xtr - mu) / sd, (Xva - mu) / sd

torch.manual_seed(0)
model = train_dl.build_model(len(train_dl.FEATURES))
w = torch.tensor((len(ytr) / (3 * np.bincount(ytr))) ** 0.75, dtype=torch.float32)   # trọng số lớp
loss_fn = torch.nn.CrossEntropyLoss(weight=w)
opt = torch.optim.Adam(model.parameters(), lr=2e-3, weight_decay=1e-5)
Xt, Yt, Xv = torch.tensor(Xtr), torch.tensor(ytr), torch.tensor(Xva)

rows = []
for epoch in range(1, 6):
    model.train()
    perm, tot = torch.randperm(len(Xt)), 0.0
    for i in range(0, len(Xt), 1024):
        idx = perm[i:i + 1024]
        opt.zero_grad()
        loss = loss_fn(model(Xt[idx]), Yt[idx])
        loss.backward()
        opt.step()
        tot += loss.item() * len(idx)
    model.eval()
    with torch.no_grad():
        pv = torch.softmax(model(Xv), 1).numpy()
    rows.append(dict(epoch=epoch, train_loss=round(tot / len(Xt), 4),
                     val_auc=round(roc_auc_score(yva, pv, multi_class="ovr"), 4),
                     val_macro_f1=round(f1_score(yva, pv.argmax(1), average="macro"), 4)))
display(pd.DataFrame(rows))
```

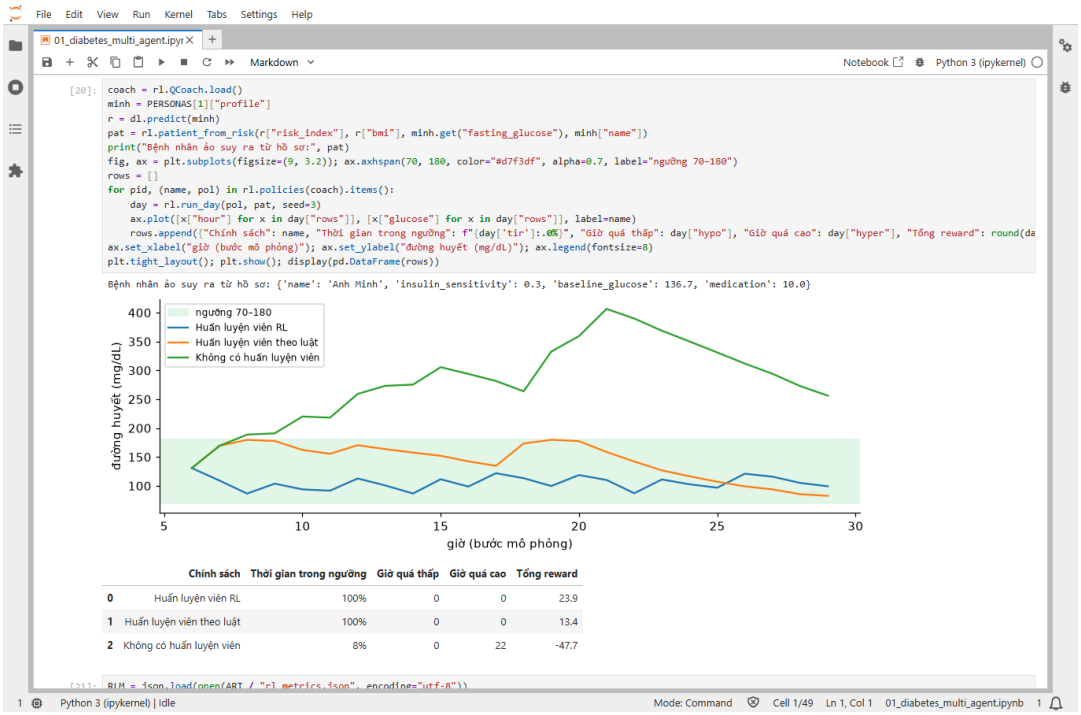
	epoch	train_loss	val_auc	val_macro_f1
0	1	0.9271	0.7804	0.4260
1	2	0.8586	0.7696	0.4402

Hình 24. Ô chạy thử thu nhỏ vòng huấn luyện MLP (5 epoch trên 40.000 hồ sơ) cùng bảng loss, AUC và macro-F1 theo epoch.



```
[11]: import networkx as nx
G = nx.DiGraph()
for nid, n in kb.nodes.items():
    G.add_node(nid, vi=n["vi"], type=n["type"])
for e in kb.edges:
    G.add_edge(e["source"], e["target"])
palette = {"condition": "#e5342d", "symptom": "#f5a623", "risk_factor": "#7060d8", "test": "#2589e0", "treatment": "#0d9f8f", "lifestyle": "#22b05a", "complication":
pos = nx.spring_layout(G.to_undirected(), seed=3, k=0.9)
fig, ax = plt.subplots(figsize=(10, 6.2)); ax.axis("off")
nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#c4d0e6", arrows=False, width=1.2)
nx.draw_networkx_nodes(G, pos, ax=ax, node_color=[palette.get(G.nodes[n]["type"], "#8896b0") for n in G], node_size=260)
nx.draw_networkx_labels(G, pos, labels={n: G.nodes[n]["vi"] for n in G}, ax=ax, font_size=6.2)
ax.legend(handles=[plt.Line2D([], [], marker="o", ls="", color=c, label=t) for t, c in palette.items()], loc="lower left", fontsize=7, frameon=False)
plt.show()
```


Hình 25. Ô vẽ đồ thị tri thức 41 nút bằng networkx, màu theo loại khái niệm.



Hình 26. Ô so ba huấn luyện viên trên bệnh nhân ảo suy ra từ hồ sơ Anh Minh: đường đường huyết trong một ngày và bảng thời gian trong ngưỡng.

5.5.1. Giải thích một vài ô quan trọng

Ô huấn luyện thu nhỏ (Phần 1.2). Vòng lặp dưới đây là lõi của việc học. Mỗi epoch xáo trộn thứ tự mẫu (randperm) rồi cắt thành các mini-batch 1.024 mẫu. Với mỗi batch, opt.zero_grad() xóa gradient cũ, loss_fn tính mất mát có trọng số lớp, loss.backward() lan truyền ngược để tính gradient, opt.step() cập nhật trọng số bằng Adam. Sau mỗi epoch, mô hình chuyển sang chế độ eval (tắt Dropout) và được đo AUC, macro-F1 trên tập validation; tập này chỉ dùng để theo dõi và chọn epoch, không dùng để cập nhật trọng số.

```
for epoch in range(1, 6):
    model.train()
    perm, tot = torch.randperm(len(Xt)), 0.0
    for i in range(0, len(Xt), 1024):
        idx = perm[i:i + 1024]
        opt.zero_grad()
        loss = loss_fn(model(Xt[idx]), Yt[idx])
        loss.backward()
        opt.step()
        tot += loss.item() * len(idx)
    model.eval()
    with torch.no_grad():
        pv = torch.softmax(model(Xv), 1).numpy()
```

Đoạn mã 7. Vòng huấn luyện trong notebook (Phần 1.2).

Ô huấn luyện Q-learning (Phần 3.3). Dòng quan trọng nhất là cập nhật giá trị Q: giá trị mới bằng giá trị cũ cộng alpha nhân sai số giữa mục tiêu (reward cộng gamma nhân giá trị tốt nhất ở trạng thái kế tiếp) và giá trị cũ. Phép max chỉ xét các hành động được phép ở giờ kế tiếp (allowed), vì ban đêm chỉ được nghỉ hoặc ăn nhẹ có đường. Hệ số epsilon giảm tuyến tính từ 1 xuống 0,05 trong 70% số ngày đầu: đầu quá trình agent thử ngẫu nhiên để khám phá, về sau chủ yếu khai thác điều đã học. Trong ô chạy thử thu nhỏ, 4.000 ngày đã gặp 1.314 trạng thái và reward trung bình tăng dần khi epsilon giảm.

```
eps = max(0.05, 1.0 - 0.95 * ep / (0.7 * episodes))
a = coach.act(st, eps, rng)                       # epsilon-greedy
nst, r, _ = step(st, a, rng)
target = r if done(nst) else r + gamma * max(
    coach.values(encode(nst))[AID[x]] for x in allowed(nst["hour"]))
v = coach.values(encode(st))
v[AID[a]] += alpha * (target - v[AID[a]])        # cập nhật Q(s, a)
```

Đoạn mã 8. Lõi của rl.train (rút gọn): chọn hành động, mô phỏng một giờ, cập nhật Q.

Ô so sánh BM25 và Graph-RAG (Phần 2.2). Với mỗi câu hỏi, notebook gọi kb.search hai lần, graph=False (BM25 thuần) và graph=True, rồi đặt đoạn xếp hạng 1 cạnh nhau cùng các nút khớp trong đồ thị. Kết quả thật: trong ba câu mẫu, Graph-RAG chỉ đổi đoạn xếp 1 ở câu về “high blood sugar” (từ tests-diagnosis:50 sang managing-diabetes:66, đoạn nói về hyperglycemia nên hợp ý hơn), hai câu còn lại giữ nguyên. Ô này cũng lộ một hạn chế: câu đó khớp nhầm nút “Huyết áp cao” vì từ high trùng với tên nút, nên khớp nút bằng từ khóa còn thô.

Ô hội chẩn (Phần 5). Clinic.consult trả về một danh sách sự kiện; notebook in thành bảng 13 dòng gồm thứ tự, agent, loại sự kiện và nội dung. Đọc bảng này từ trên xuống chính là xem lại luồng: bác sĩ trưởng chuyển lượt, bác sĩ dữ liệu ước lượng nguy cơ, thủ thư tri thức trích dẫn, hộ lý an toàn cảnh báo, huấn luyện viên gợi ý ngày sinh hoạt, thư ký báo cáo vẽ hình và bảng, rồi bác sĩ trưởng kết luận.

5.6. Nâng cấp demo: phản biện, giải thích, hỏi đáp, độ tin cậy

Phiên bản này bổ sung tám khả năng cho phòng khám 3D. Hệ thống có thêm agent thứ bảy là Bác sĩ phản biện (ngồi cạnh Thủ thư tri thức trong phòng khám); sơ đồ ở Chương 3 vẫn vẽ sáu agent chuyên môn. Mọi con số dưới đây được tính từ tập kiểm tra 50.736 hồ sơ hoặc từ các bộ câu hỏi có đáp án, không có số liệu nào được nhập tay.

5.6.1. Bác sĩ phản biện và tranh luận

Mô hình MLP không nhận đường huyết làm đầu vào, nên một người có đường huyết đói 190 mg/dL nhưng hồ sơ lối sống lành mạnh (vai Cô Hoa) vẫn được mô hình báo nguy cơ thấp. Bác sĩ phản biện (mas/critic.py) kiểm tra chéo kết luận đó bằng bằng chứng chạy được, độc lập với mô hình: luật an toàn cùng ngưỡng ADA, số đo thật của hồ sơ, phép thử độ nhạy (chạy lại mô hình khi cân nặng lệch ±5 kg) và đoạn trích có nguồn từ kho 492 đoạn NIDDK/CDC/MedlinePlus. Khi có mâu thuẫn thật, các agent tranh luận tối đa hai vòng: vòng 1 bác sĩ phản biện nêu bằng chứng và bác sĩ dữ liệu bảo vệ; vòng 2 hộ lý an toàn nêu luật, thủ thư tri thức đọc nguyên văn tài liệu, bác sĩ phản biện kết luận. Nguyên tắc kết luận là ưu tiên số đo trực tiếp: mức nguy cơ của Cô Hoa được nâng từ thấp lên cao kèm đề nghị xét nghiệm xác

nhận. Các lượt tranh luận hiện thành bong bóng hội thoại 3D và được đọc bằng giọng nói riêng của từng agent.

Bảng 13. Bốn phép thử của Bác sĩ phản biện.

Phép thử	Bằng chứng dùng	Khi nào coi là mâu thuẫn
C1 Đường huyết so với mô hình	Đường huyết đói, luật R1–R4, ngưỡng ADA	Đường huyết ≥ 126 mg/dL mà mô hình không báo nguy cơ cao; hoặc 100–125 mà mô hình báo thấp
C2 Triệu chứng so với mô hình	Triệu chứng điển hình của NIDDK	Từ hai triệu chứng điển hình trở lên mà mô hình báo thấp
C3 Độ chắc chắn	Xác suất ba nhóm của mô hình	Hai nhóm đứng đầu cách nhau dưới 10 điểm phần trăm
C4 Độ nhạy với sai số đo	Chạy lại mô hình, cân nặng ±5 kg	Mức nguy cơ đổi khi cân nặng lệch nhẹ



Hình 27. Tranh luận vòng 2 trong phòng khám 3D: hộ lý an toàn nêu luật R3, bác sĩ dữ liệu và bác sĩ phản biện đáp lại.



Hình 28. Thẻ phản biện (trái) và thẻ giải thích với what-if tương tác (phải).

5.6.2. Giải thích kết quả

Với mỗi bệnh nhân, giao diện vẽ các yếu tố đẩy nguy cơ lên (đỏ) hoặc kéo xuống (xanh), tính bằng cách thay từng yếu tố bằng giá trị trung bình dân số rồi chạy lại mô hình. Phần what-if cho phép kéo cân nặng hoặc bật tắt hút thuốc, vận động, trái cây, rau, rượu bia; mô hình tính lại ngay và vẽ đường cong nguy cơ theo BMI. Nút nghe giải thích đọc kết quả bằng giọng nói. Đây là thay đổi theo mô hình học từ khảo sát, không phải lời hứa về hiệu quả với từng người.

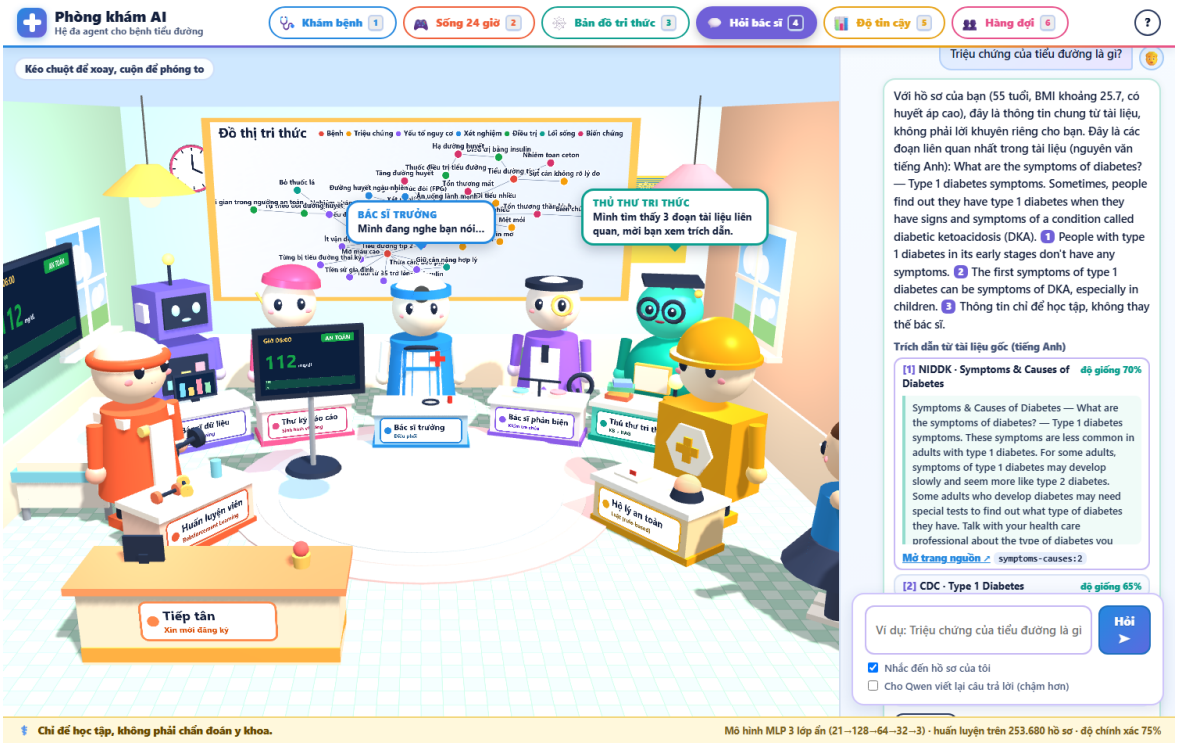
5.6.3. Hỏi bác sĩ bằng câu hỏi tự do

Ô chat trả lời kèm trích dẫn [1], [2] từ kho 492 đoạn. Truy hồi kết hợp BM25, đồ thị tri thức và vector MiniLM; câu tiếng Việt đi qua bảng thuật ngữ Việt–Anh. Hệ thống từ chối khi câu hỏi cần chẩn đoán hay liều thuốc cá nhân, khi có dấu hiệu cấp cứu, hoặc khi không có đoạn đủ liên quan. Qwen2.5-1.5B-Instruct chạy được nhưng mất 40–140 giây trên máy thiếu bộ nhớ ảo và bộ kiểm tra bám tài liệu thường loại câu sinh ra, nên chế độ mặc định là câu trả lời trích đoạn; giao diện luôn ghi rõ đang dùng chế độ nào.

Bảng 14. Truy hồi trích đoạn trên các câu hỏi có đáp án (cận dưới vì chỉ tính đoạn đã gắn tay).

Bộ câu hỏi	Số câu	Đúng ở vị trí 1	Đúng trong 3 đoạn đầu
Tiếng Anh (diabetes_qa_demo.json)	40	16/40 (40,0%)	30/40 (75,0%)
Tiếng Việt (12 câu dịch tay)	12	2/12 (16,7%)	7/12 (58,3%)

Gộp	52	18/52 (34,6%)	37/52 (71,2%)



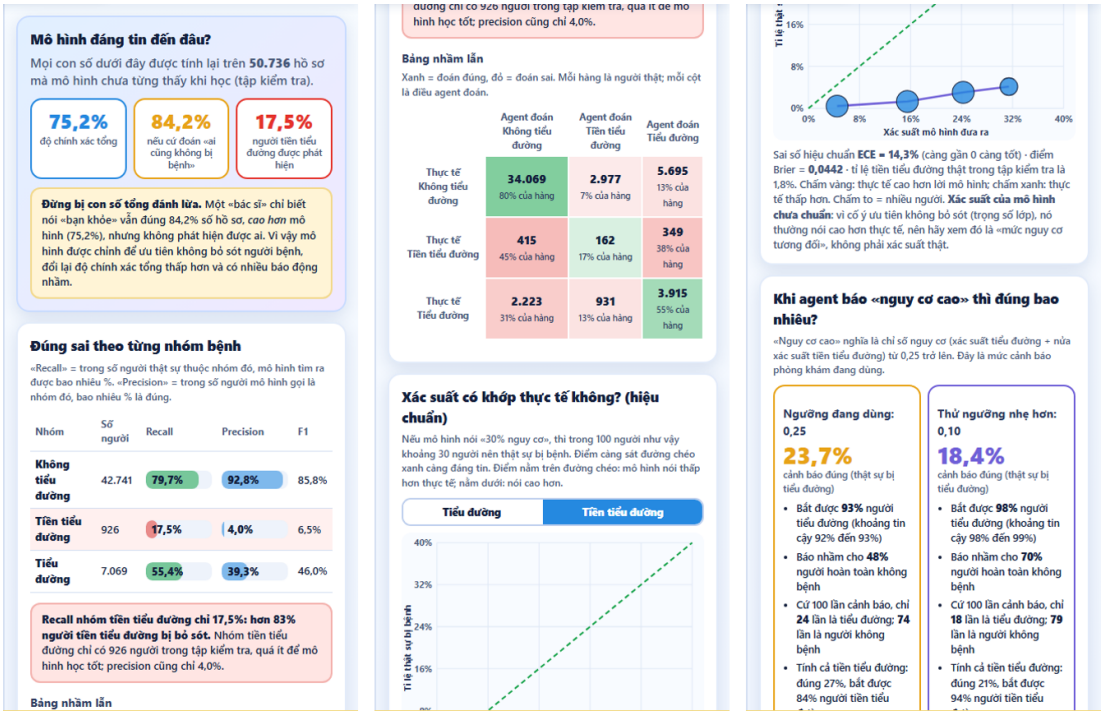
Hình 29. Tab Hỏi bác sĩ: câu trả lời kèm trích dẫn có liên kết tới trang nguồn; thủ thư tri thức 3D thông báo kết quả.

5.6.4. Độ tin cậy và kiểm tra công bằng

Tab Độ tin cậy trình bày kết quả của project01/train/eval_trust.py trên tập kiểm tra: độ chính xác từng nhóm, recall nhóm tiền tiểu đường, biểu đồ hiệu chuẩn xác suất, tỉ lệ cảnh báo đúng và độ chính xác theo nhóm tuổi, giới, thu nhập (có khoảng tin cậy Wilson 95%). Tab nói thẳng điểm yếu: độ chính xác tổng thấp hơn baseline luôn đoán không tiểu đường, nhóm tiền tiểu đường gần như bị bỏ sót, và người trẻ cùng người thu nhập cao bị bỏ sót nhiều hơn.

Bảng 15. Các số liệu độ tin cậy chính trên tập kiểm tra 50.736 hồ sơ.

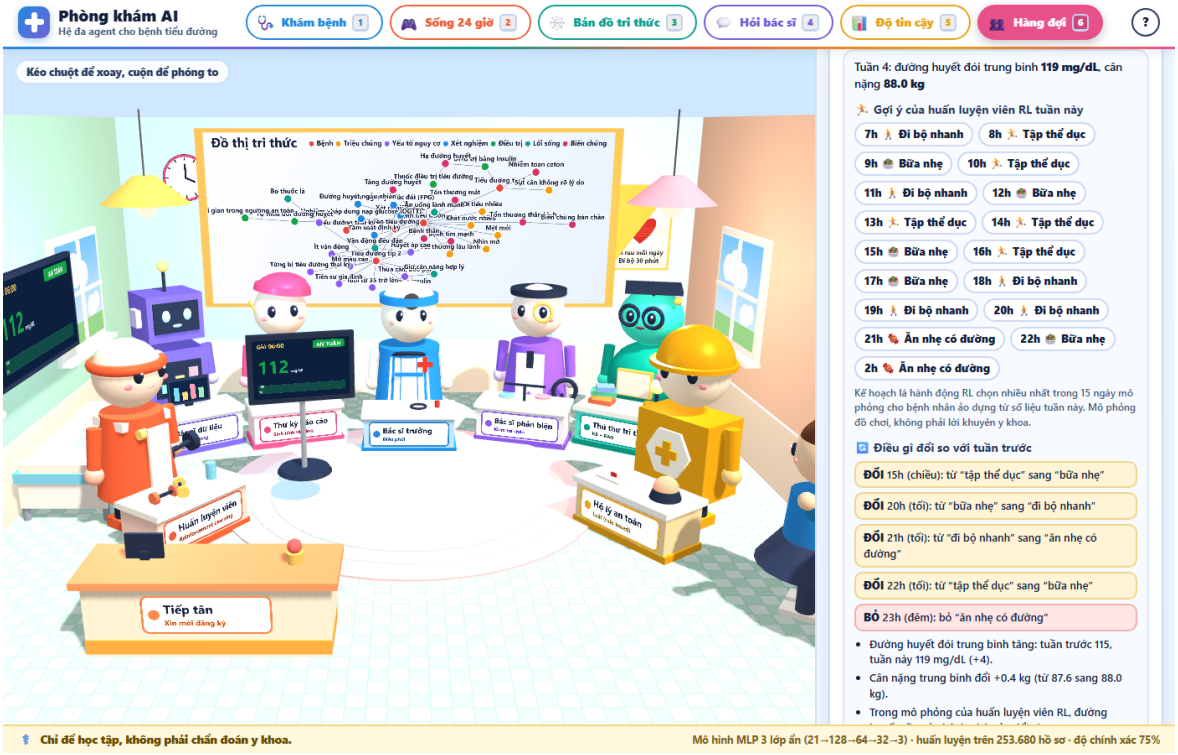
Chỉ số	Giá trị
Độ chính xác tổng (baseline luôn đoán không tiểu đường)	75,2% (84,2%)
Recall: không / tiền / tiểu đường	79,7% / 17,5% / 55,4%
Precision nhóm tiền tiểu đường	4,0%
Cảnh báo nguy cơ cao (chỉ số ≥ 0,25): tỉ lệ bị cảnh báo	54,4%
Trong số được cảnh báo, thật sự tiểu đường	23,7%
Ca tiểu đường được cảnh báo (recall) / người không bệnh bị báo nhầm	92,5% / 47,5%
Chênh lệch recall tiểu đường giữa các nhóm tuổi / thu nhập / giới (điểm %)	49,2 / 24,9 / 3,5



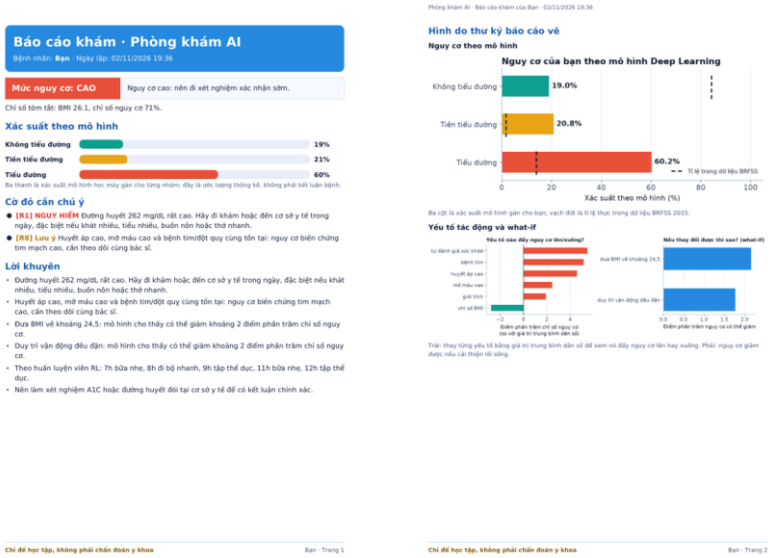
Hình 30. Tab Độ tin cậy: tổng quan, ma trận nhầm lẫn, biểu đồ hiệu chuẩn và kiểm tra công bằng.

5.6.5. Hàng đợi, xu hướng 4 tuần, báo cáo PDF và giọng nói

Hàng đợi sắp xếp các hồ sơ theo mức khẩn (cờ đỏ trước, rồi chỉ số nguy cơ giảm dần) kèm lý do. Xu hướng 4 tuần theo dõi đường huyết đói và cân nặng; mỗi tuần huấn luyện viên RL dựng lại bệnh nhân từ số đo của tuần đó và đưa ra kế hoạch ngày, giao diện chỉ ra giờ nào đổi so với tuần trước. Chuỗi 28 ngày trong demo là dữ liệu mẫu sinh bằng seed cố định, được ghi rõ trên giao diện, không phải số đo thật. Nút tải PDF ghép hình và bảng của thư ký báo cáo thành báo cáo cho bệnh nhân. Nhập liệu bằng giọng nói dùng Azure Speech-to-Text với cùng khóa của TTS (khóa chỉ nằm ở máy chủ) và bộ phân tích luật để tự điền tuổi, giới, chiều cao, cân nặng, đường huyết, bệnh nền và triệu chứng; vòng thử tạo tiếng bằng Azure TTS rồi nhận dạng lại cho kết quả đúng.



Hình 31. Theo dõi 4 tuần (dữ liệu mẫu): kế hoạch của huấn luyện viên RL ở tuần 4 và các giờ đã đổi so với tuần 3.



Hình 32. Hai trang đầu của báo cáo PDF cho bệnh nhân.

Bộ kiểm thử mở rộng thêm các tệp test_project01_critic, _trust, _ask, _stt, _followup và _pdf; toàn bộ 64 kiểm thử của Project 01 đều đạt. Notebook được bổ sung các Phần 5b–5f cho các tính năng trên (64 ô). Điểm yếu còn lại: Qwen chưa dùng được ổn định trên máy này; hit@3 của câu hỏi tiếng Việt mới đạt 58,3%; dữ liệu 4 tuần là mô phỏng; mô hình vẫn chỉ phản ánh tương quan trong khảo sát Hoa Kỳ năm 2015.

5.7. Kiểm thử

Bộ kiểm thử tự động (tests/test_project01.py) gồm 8 kiểm thử cốt lõi (cùng các tệp kiểm thử bổ sung cho tính năng mới, mục 5.6) cho Project 01: lược đồ hồ sơ khớp 21 chỉ số BRFSS; xác suất cộng bằng 1 và bệnh nhân rủi ro cao có chỉ số lớn hơn; luật R1 kích hoạt với đường huyết nguy hiểm; đồ thị nhất quán và RAG trích dẫn đúng nguồn NIDDK; huấn luyện viên RL tốt hơn không có huấn luyện viên; giới hạn hành động ban đêm; kịch bản hội chẩn đủ sáu agent, kết thúc bằng sự kiện final, có ba hình báo cáo; và làm sạch văn bản trước khi đọc thành tiếng. Cả tám kiểm thử đều qua.

5.8. Đánh giá và hạn chế

- Mô hình Deep Learning chỉ đạt AUC 0,78 và gần như không phân biệt được tiền tiểu đường: dữ liệu khảo sát không có xét nghiệm máu nên giới hạn dự đoán là có thật.
- Dữ liệu BRFSS là khảo sát tự khai ở Hoa Kỳ năm 2015, không đại diện cho mọi quần thể và chỉ phản ánh tương quan.
- Đồ thị tri thức nhỏ (41 nút) do nhóm biên soạn; tài liệu nguồn bằng tiếng Anh; bộ đánh giá RAG chỉ có 16 câu.
- Mô phỏng 24 giờ là mô hình đồ chơi, tham số đặt theo trực giác; kết quả RL gần tối đa chỉ có ý nghĩa trong môi trường này.
- Hệ thống chỉ để học tập, không phải thiết bị hay lời khuyên y tế.

5.9. Hướng phát triển

- Dùng nhiều năm BRFSS hoặc dữ liệu có xét nghiệm (A1C) để cải thiện dự đoán nhóm tiền tiểu đường.
- Nạp đồ thị vào Neo4j thật và mở rộng đồ thị từ nhiều nguồn y khoa bằng cách trích xuất quan hệ.
- Thay mô phỏng đồ chơi bằng mô phỏng glucose–insulin đã được kiểm định; thử RL sâu thay cho bảng Q.
- Thêm agent tiếng Việt cho câu hỏi tự do (kết hợp mô hình ngôn ngữ nhỏ chạy cục bộ) nhưng giữ Hộ lý an toàn kiểm soát đầu ra.

6. Kết luận

Project đã xây dựng một hệ đa agent hoàn chỉnh cho bệnh tiểu đường, bám theo mô hình đề bài: dữ liệu có sẵn qua Deep Learning để dự đoán và phân loại; tài liệu qua Knowledge Base và RAG; huấn luyện viên kết hợp reinforcement learning và luật; tất cả hội tụ ở Bác sĩ trưởng để tư vấn và hỗ trợ. Kết quả định lượng: MLP đạt AUC 0,78 trên 50.736 hồ sơ kiểm tra; Graph-RAG đạt hit@3 0,62 so với 0,56 của BM25 thuần; huấn luyện viên RL giữ đường huyết trong ngưỡng an toàn gần như toàn thời gian trong môi trường mô phỏng.

Điều rút ra quan trọng nhất không nằm ở các con số, mà ở cách thiết kế: tách nhiệm vụ thành các agent nhỏ, mỗi agent có đầu ra có cấu trúc, và luôn để một agent luật độc lập kiểm soát an toàn. Demo phòng khám 3D cho thấy cách làm này có thể giải thích được từng bước với người dùng không chuyên.

Tài liệu tham khảo

[1] Google Cloud. What are AI agents? Definition, examples, and types. https://cloud.google.com/discover/what-are-ai-agents

[2] IBM, A. Gutowska. What Are AI Agents? https://www.ibm.com/think/topics/ai-agents

[3] McKinsey & Company. What is an AI agent? 25/03/2025. https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-an-ai-agent

[4] NIDDK. What is diabetes?; Symptoms & causes of diabetes; Diabetes tests and diagnosis; Managing diabetes. National Institute of Diabetes and Digestive and Kidney Diseases, https://www.niddk.nih.gov/health-information/diabetes (truy cập 13/10/2026).

[5] Yao, S. và cộng sự. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.

[6] Lewis, P. và cộng sự. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[7] Russell, S. và Norvig, P. Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020.

[8] Centers for Disease Control and Prevention. Behavioral Risk Factor Surveillance System (BRFSS) 2015. https://www.cdc.gov/brfss/

[9] American Diabetes Association Professional Practice Committee. Standards of medical care in diabetes—2022. Diabetes Care 45(suppl 1):S1–S16, 2022.

[10] Watkins, C. J. C. H. và Dayan, P. Q-learning. Machine Learning 8, 279–292, 1992.

[11] Robertson, S. và Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 3(4), 2009.

[12] Three.js r128. https://threejs.org/ (thư viện đồ họa 3D chạy trên trình duyệt).